# 18-740: Modern Computer Architecture & Design

# Lecture 8: "Modern Memory Dataflow and Data Storage"

Olivia Hsu

September 17, 2026

- Required Reading Assignment:
  - **Chapter 3 of Shen and Lipasti (SnL).**
  - From **Synthesis lectures of Computer Architecture**
    - Sec. 1 & 3: Bruce Jacob, "The Memory System: You Can't Avoid It, You Can't Ignore It, You Can't Fake It," 2009.
    - Sec. 1 & 4: Balasubramonian Rajeev, "Innovations in the Memory System," 2019.
- **Lab 2** due 10/9/2026, start early!
- **HW 1** due 9/18/2026

Electrical & Computer ENGINEERING

# 18-740: Modern Computer Architecture & Design

## Lecture 8:
## "Modern Memory Dataflow and Data Storage"

A. Long Latency Memory Instructions

B. Why Should we Care about Memory (MLP)

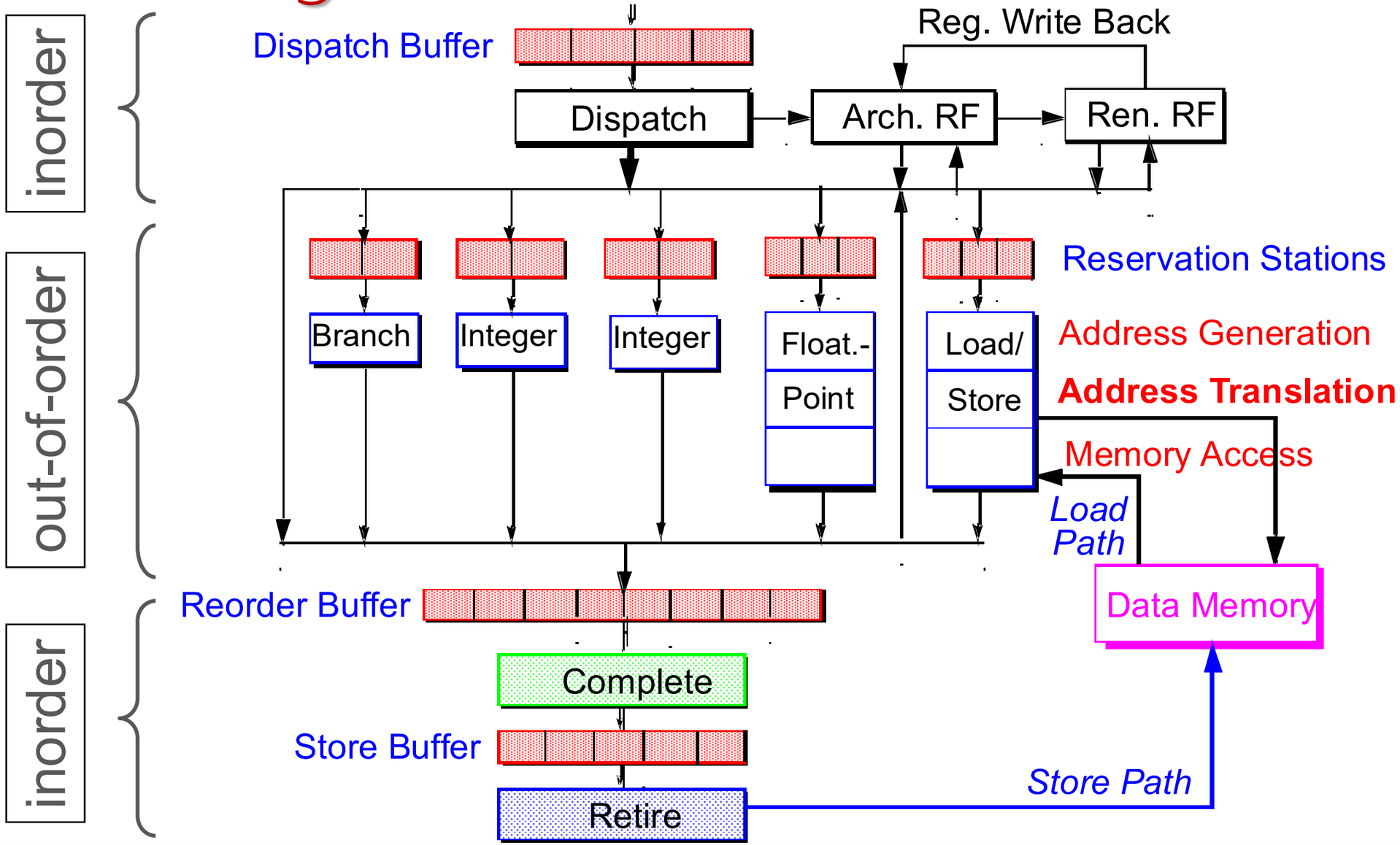
C. Modern Memory Hierarchy and Data Storage (DRAM)

Electrical & Computer ENGINEERING

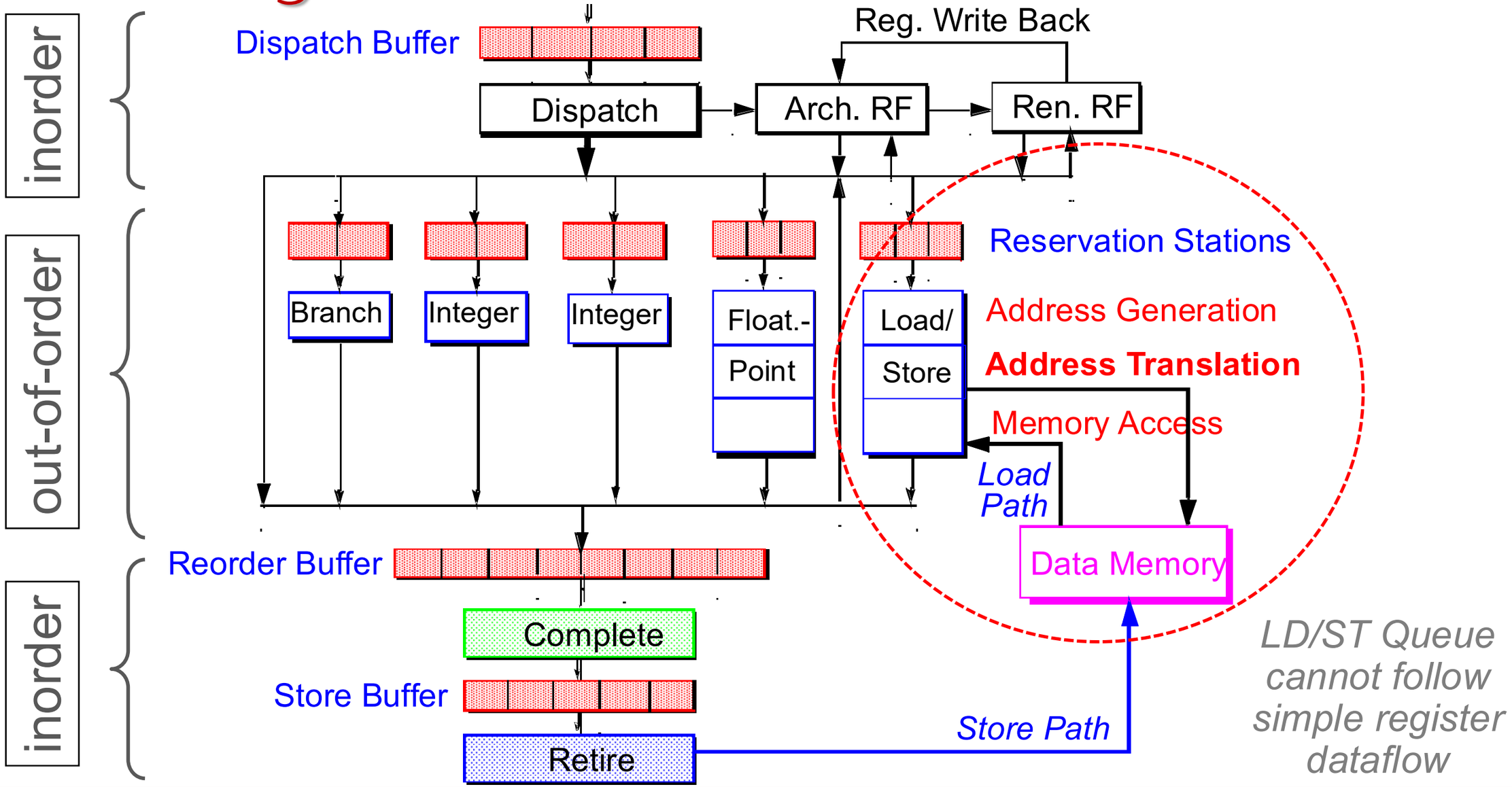
# Total Ordering of Loads and Stores

- Keep all loads and stores totally in order with respect to each other.
- However, loads and stores can execute out of order with respect to other types of instructions (while obeying register data dependencies).
- Consequently, stores are held for all previous instructions, and loads are held for stores.
  - I.e. stores performed at commit point
  - Sufficient to prevent wrong branch path stores since all prior branches now resolved

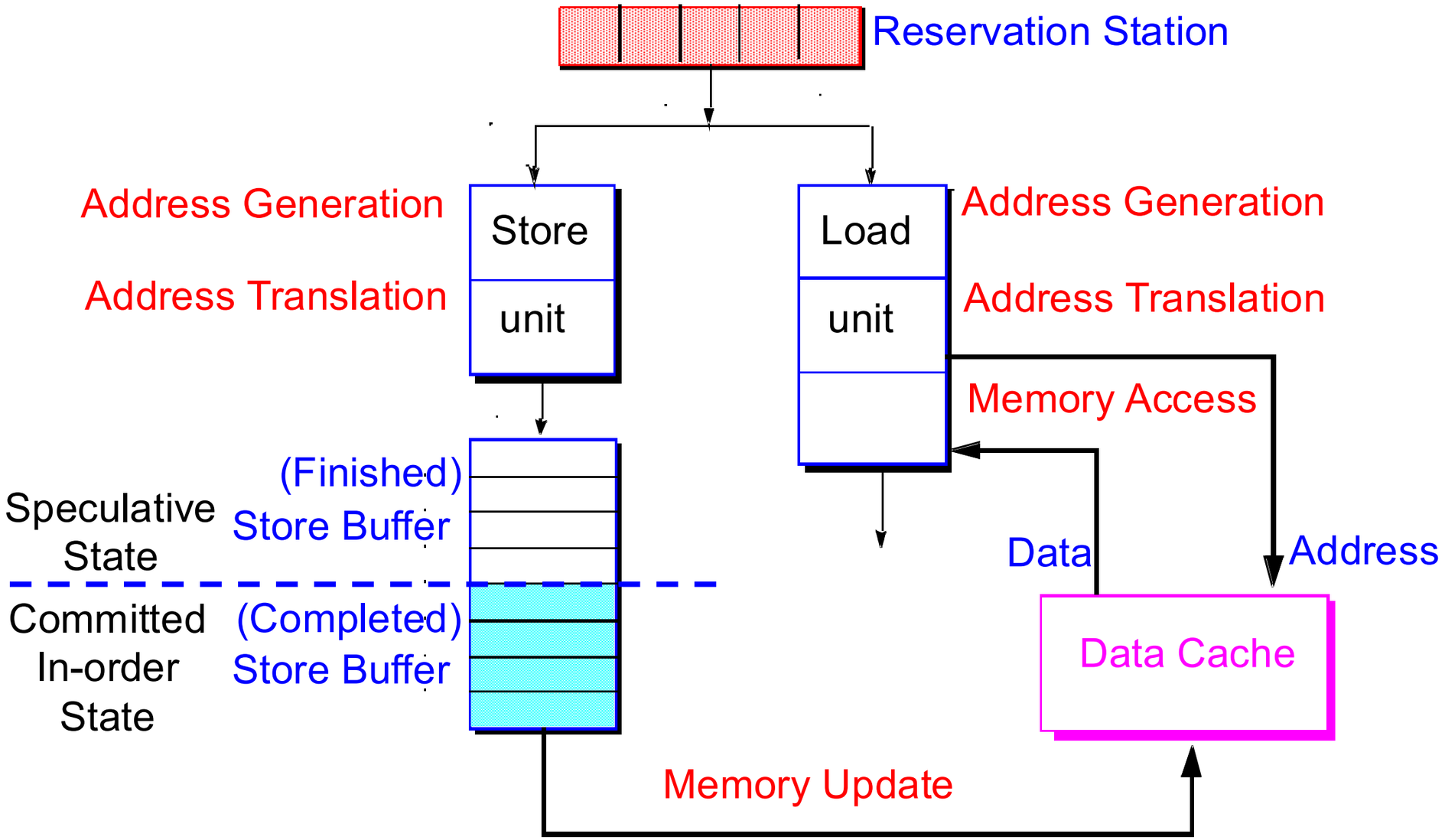
# Processing of Load/Store Instructions



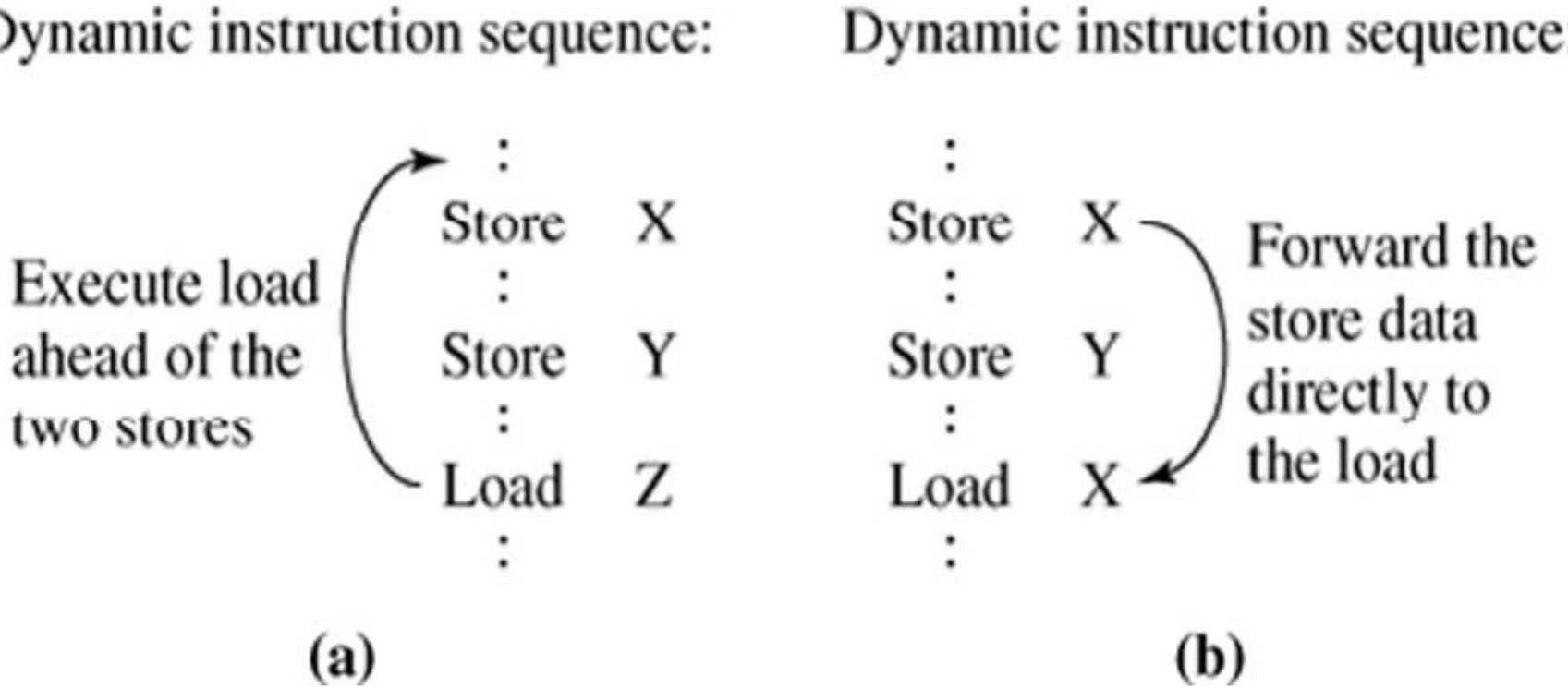
# Processing of Load/Store Instructions



# Load/Store Units and Store Buffer



# Load Bypassing & Load Forwarding: Motivation

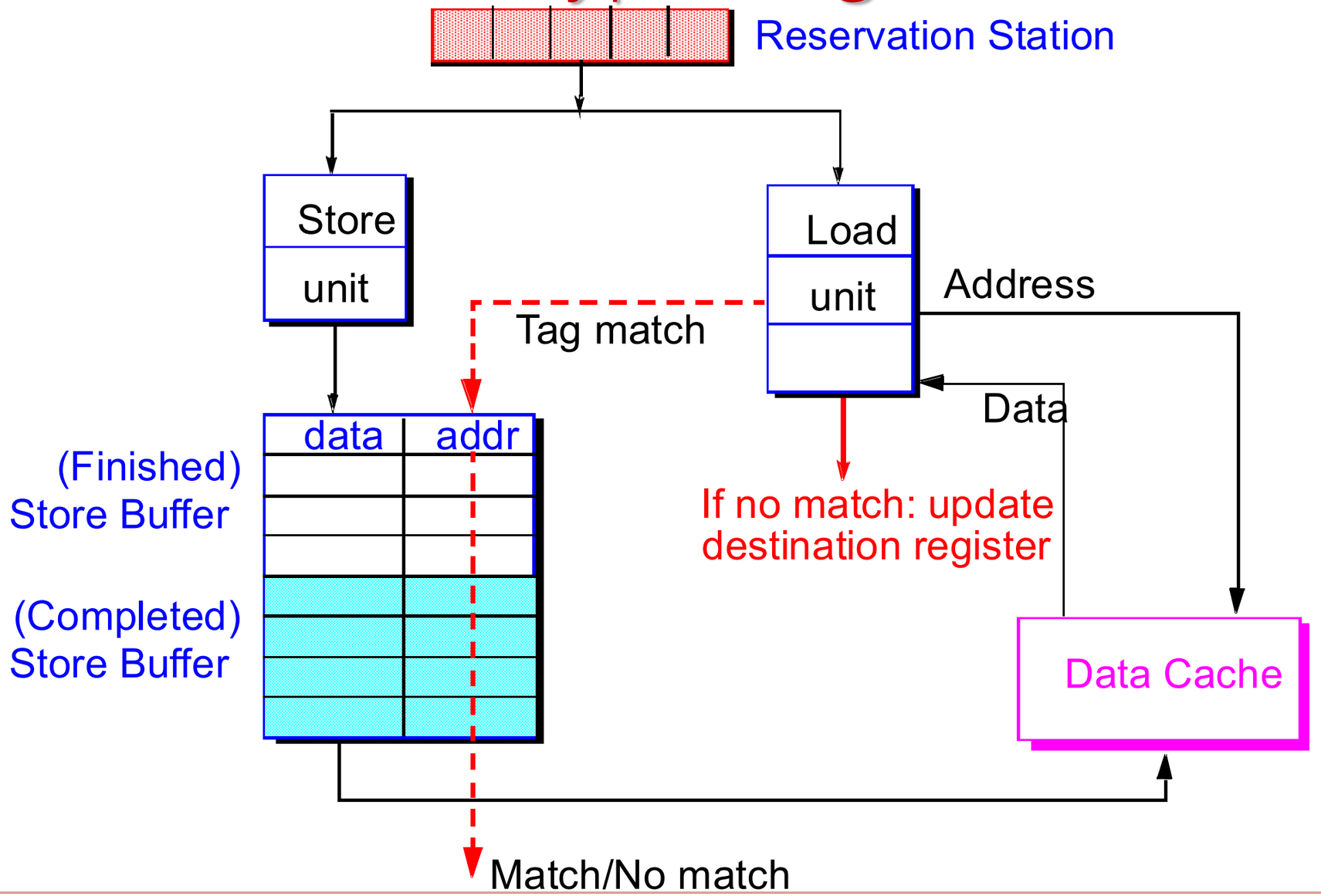


# B. Load Bypassing

- Loads can be allowed to bypass older stores if no aliasing is found
  - Older stores' addresses must be computed before loads can be issued to allow checking for RAW load dependences. If dependence cannot be checked, e.g. store address cannot be determined, then all subsequent loads are held until address is valid (conservative).
- Alternatively a load can assume no aliasing and bypass older stores *speculatively*
  - Validation of no aliasing with previous stores must be done and mechanism for reversing the effect must be provided.
- Stores are kept in ROB until all previous instructions complete, and kept in the store buffer until gaining access to cache port.
  - At completion time, a store is moved to the Completed Store Buffer to wait for turn to access cache. Store buffer is "future file" for memory.

*Store is consider completed. Latency beyond this point has little effect on the processor throughput.*

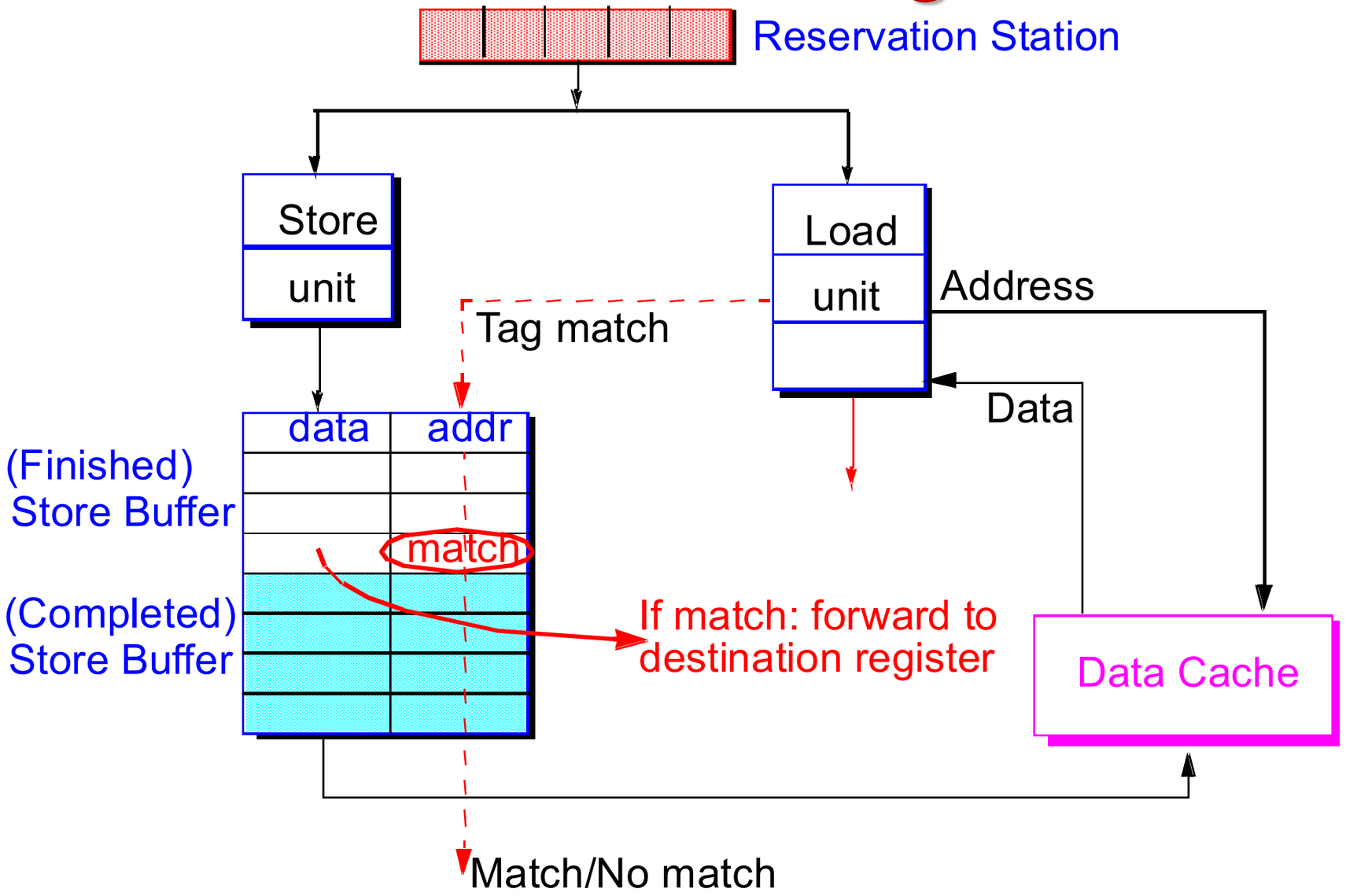
# Illustration of Load Bypassing



# C. Load Forwarding

- If a pending load is RAW dependent on an earlier store still in the store buffer, it need not wait till the store is issued to the data cache
- The load can be directly satisfied from the store buffer if both load and store addresses are valid and the data is available in the store buffer
- Since data is sourced directly from the store buffer, this avoids the latency (and power consumption) of accessing the data cache

# Illustration of Load Forwarding

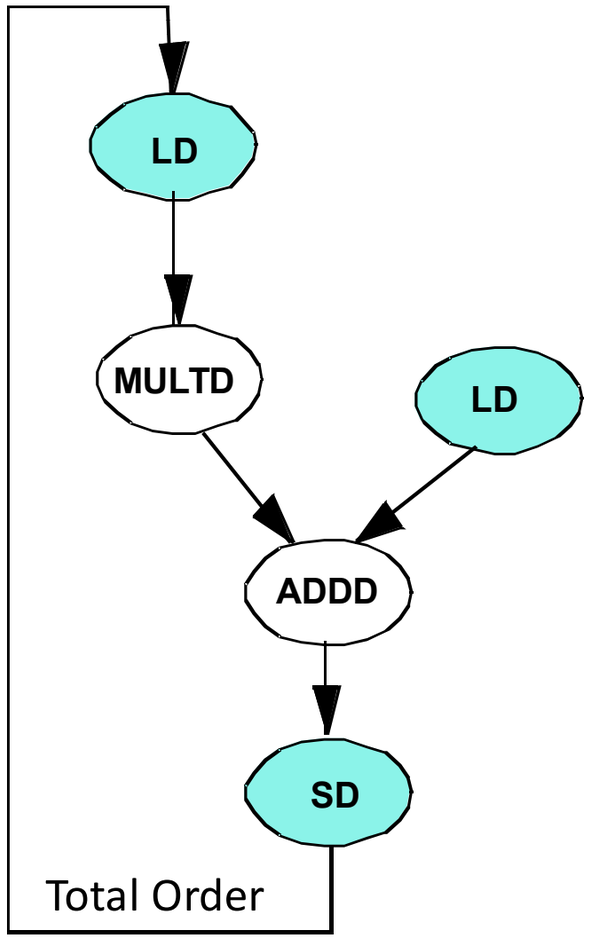


# The "DAXPY" Example

Y(i) = A * X(i) + Y(i)

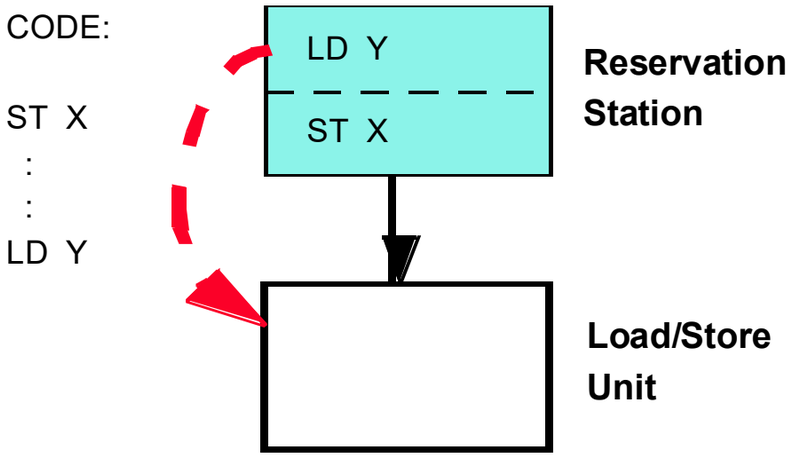
```
        LD      F0, a
        ADDI    R4, Rx, #512        ; last address

Loop:
        LD      F2, 0(Rx)           ; load X(i)
        MULTD   F2, F0, F2          ; A*X(i)
        LD      F4, 0(Ry)           ; load Y(i)
        ADDD    F4, F2, F4          ; A*X(i) + Y(i)
        SD      F4, 0(Ry)           ; store into Y(i)
        ADDI    Rx, Rx, #8          ; inc. index to X
        ADDI    Ry, Ry, #8          ; inc. index to Y
        SUB     R20, R4, Rx         ; compute bound
        BNZ     R20, loop           ; check if done
```

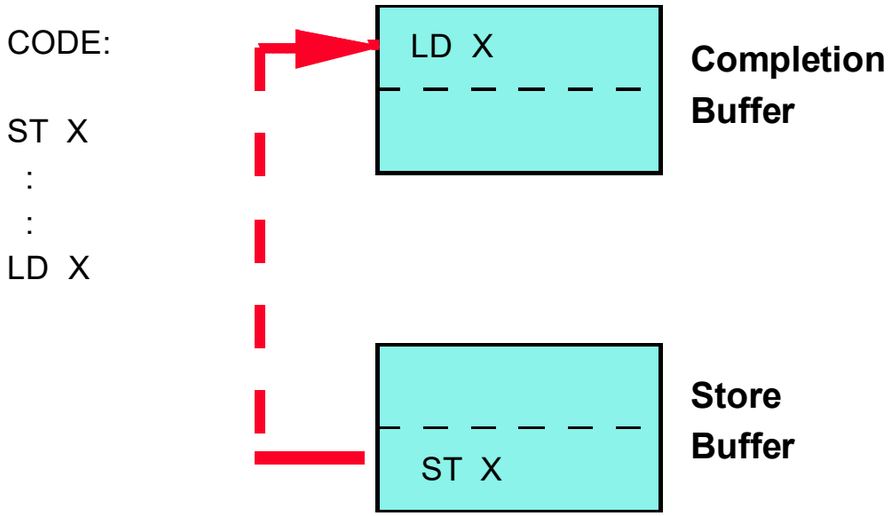


# Performance Gains From Weak Ordering

## Load Bypassing:




## Load Forwarding:




## Performance gain:

Load bypassing: 11%-19% increase over total ordering

Load forwarding: 1%-4% increase over just load bypassing

# Optimizing Load/Store Disambiguation

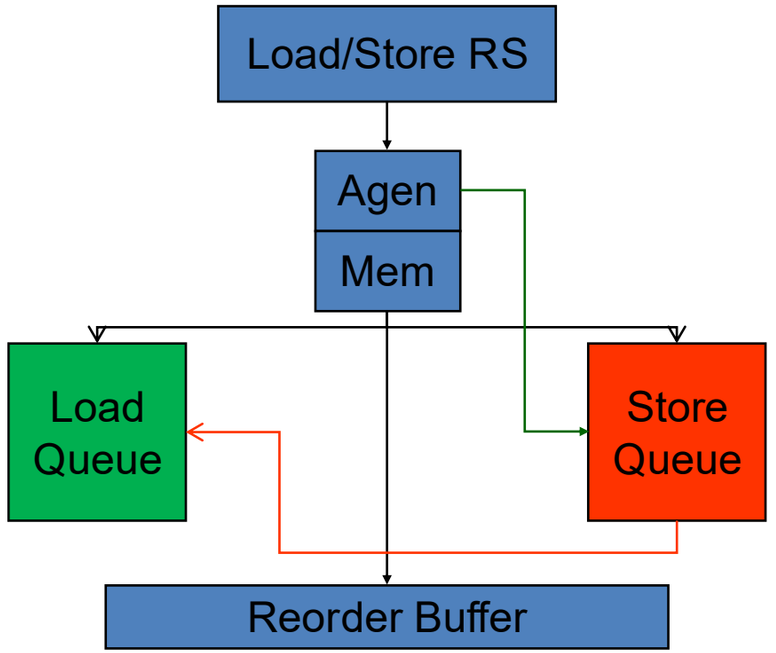
- Non-speculative load/store disambiguation
  1. Loads wait for addresses of all prior stores
  2. Full address comparison
  3. Bypass if no match, forward if match

- (1.) can limit performance:

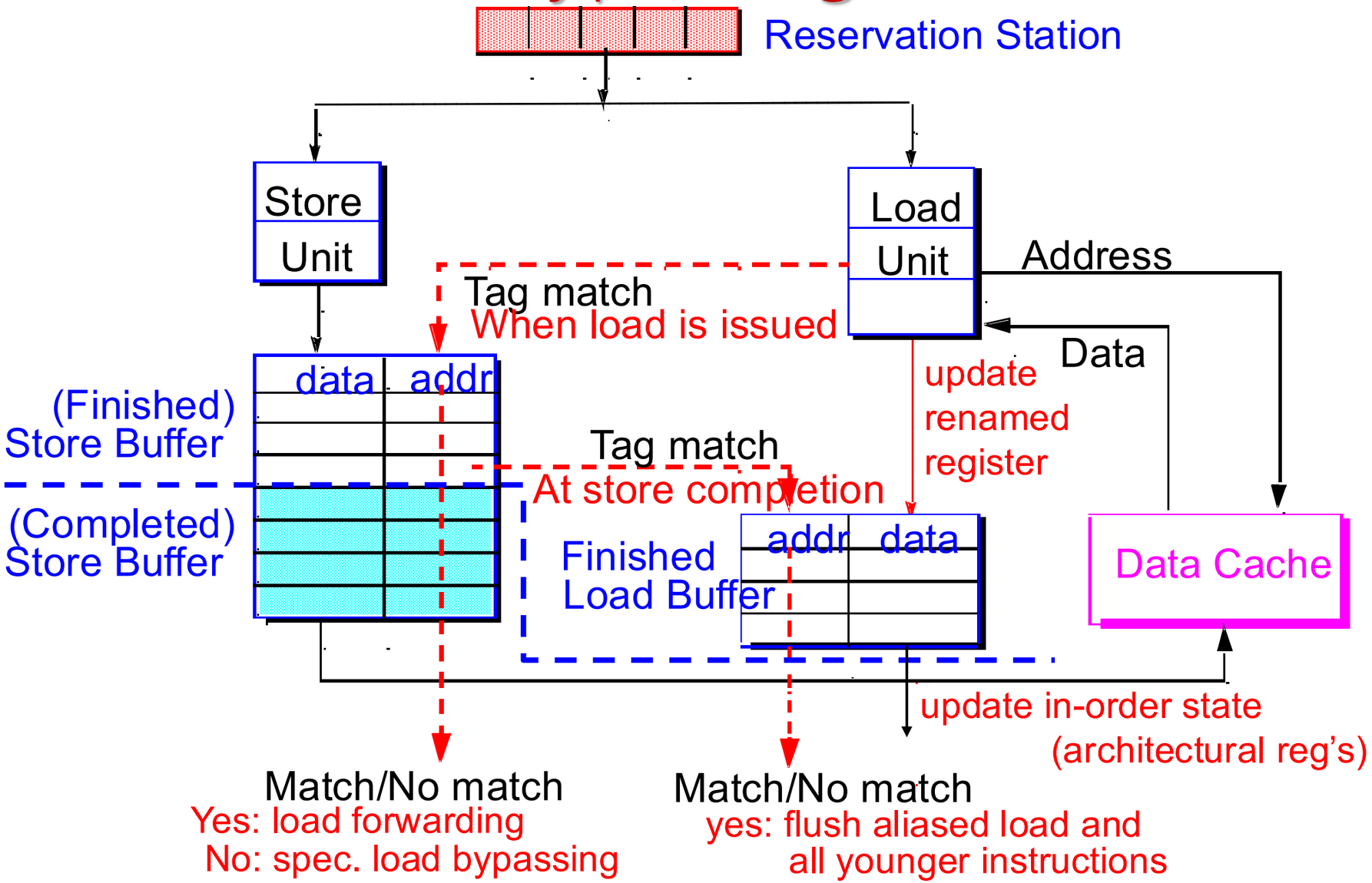
```
load    r5,MEM[r3]  ← cache miss
store   r7, MEM[r5] ← RAW for agen (addr. Gen.), stalled
…
load    r8, MEM[r9] ← independent load stalled
```

# D. Speculative Disambiguation

- What if aliases are rare?
  1. Loads don't need to wait for addresses of all prior stores
  2. Full address comparison of stores that are ready
  3. Bypass if no match, forward if match
  4. Check all store addresses when they commit
     - No matching loads – speculation was correct
     - Matching un-bypassed load – incorrect speculation
  5. Replay starting from incorrect load



# Speculative Load Bypassing



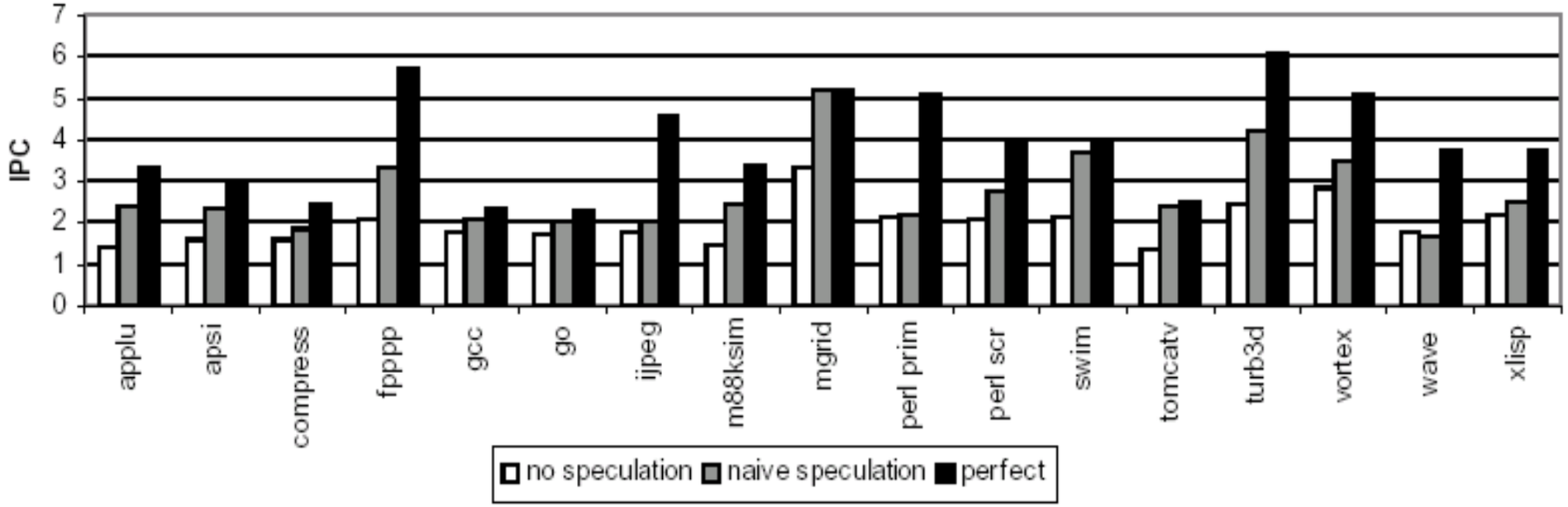
# Use of Memory Alias Prediction

- If aliases are rare: static prediction
  - Predict no alias every time
    - Why even implement forwarding? PowerPC 620 doesn't
  - Pay mis-prediction penalty rarely
- If aliases are more frequent: dynamic prediction
  - Use PHT-like history table for loads
    - If alias predicted: delay load
    - If aliased pair predicted: forward from store to load
      - More difficult to predict pair [store sets, Alpha 21264]
  - Pay mis-prediction penalty rarely
- Memory cloaking [Moshovos, Sohi]
  - Predict load/store pair
  - Directly copy store data register to load target register
  - Reduce data transfer latency to absolute minimum

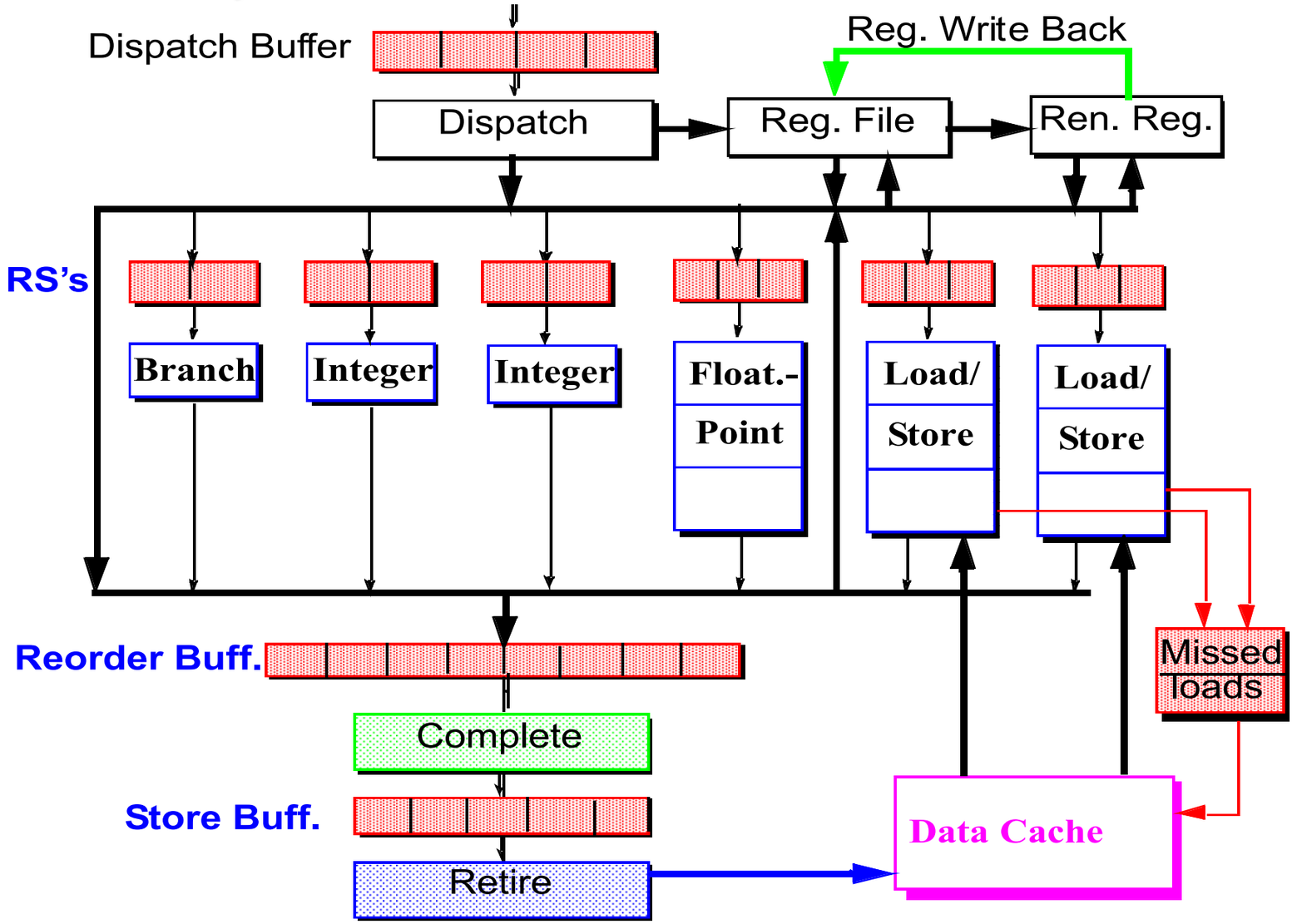
# Motivation for Selective Speculation

- Speculation options when store address not available
  - Pessimistic: always assume dependence is there
  - Naïve: always assume there is no dependence
  - Selective: use a scheme to select profitable loads to speculate
- What is the effect of dependence misprediction?

Figure 3.1: Performance of No Speculation, Naive Speculation and Perfect Prediction



# E. The Memory Bottleneck (missed-load buffer)



# Memory Bottleneck Techniques

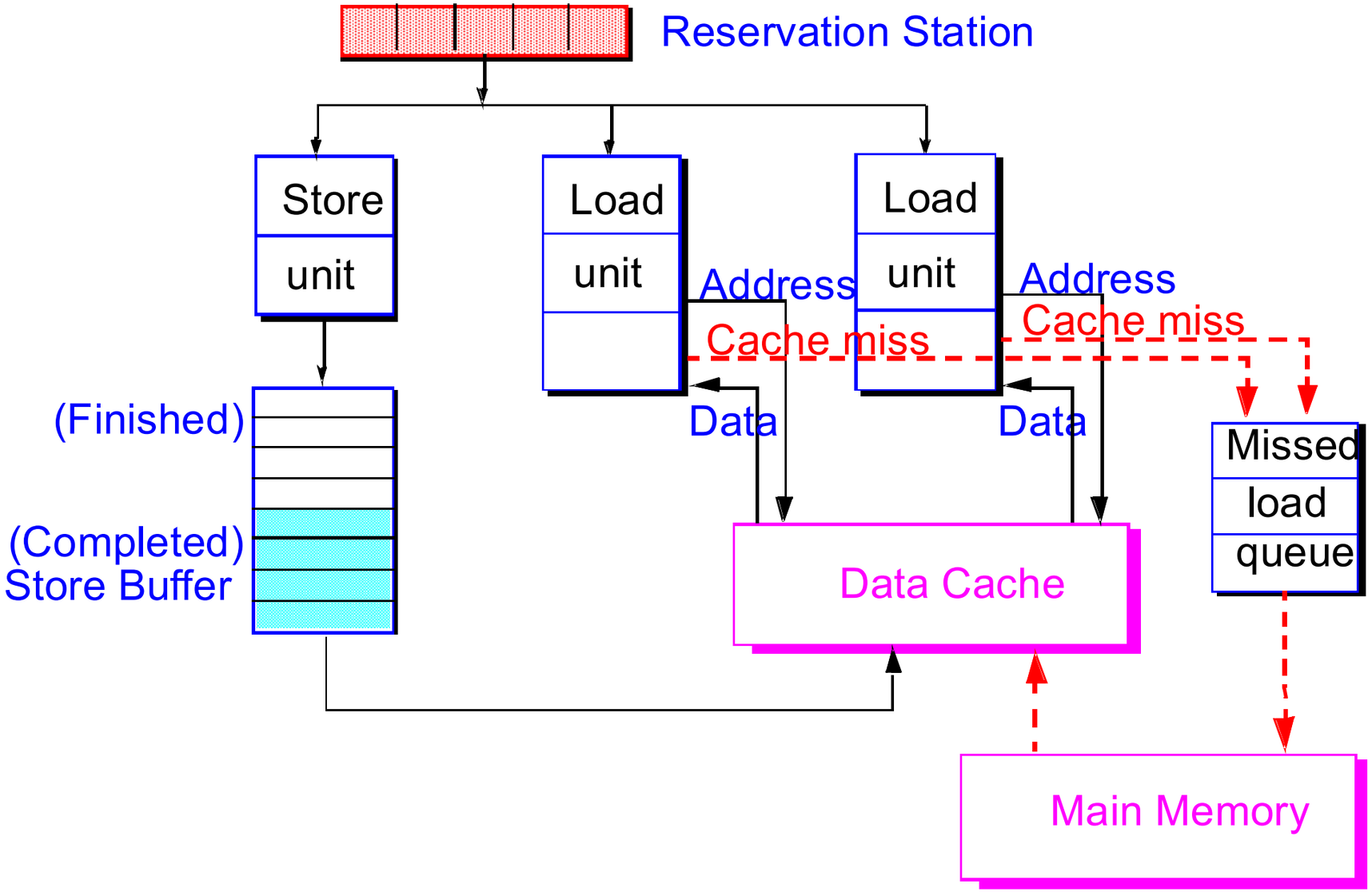
## Dynamic Hardware (Microarchitecture):

- Use Multiple Load/Store Units (need multiported D-cache)
- Use More Advanced Caches (victim cache, stream buffer)
- Use Hardware Prefetching (need load history and stride detection)
- Use Non-blocking D-cache (need missed-load buffers/MSHRs)
- Large instruction window (memory-level parallelism)

## Static Software (Code Transformation):

- Insert Prefetch or Cache-Touch Instructions (mask miss penalty)
- Array Blocking Based on Cache Organization (minimize misses)
- Reduce Unnecessary Load/Store Instructions (redundant loads)
- Software Controlled Memory Hierarchy (expose it to above DSI)

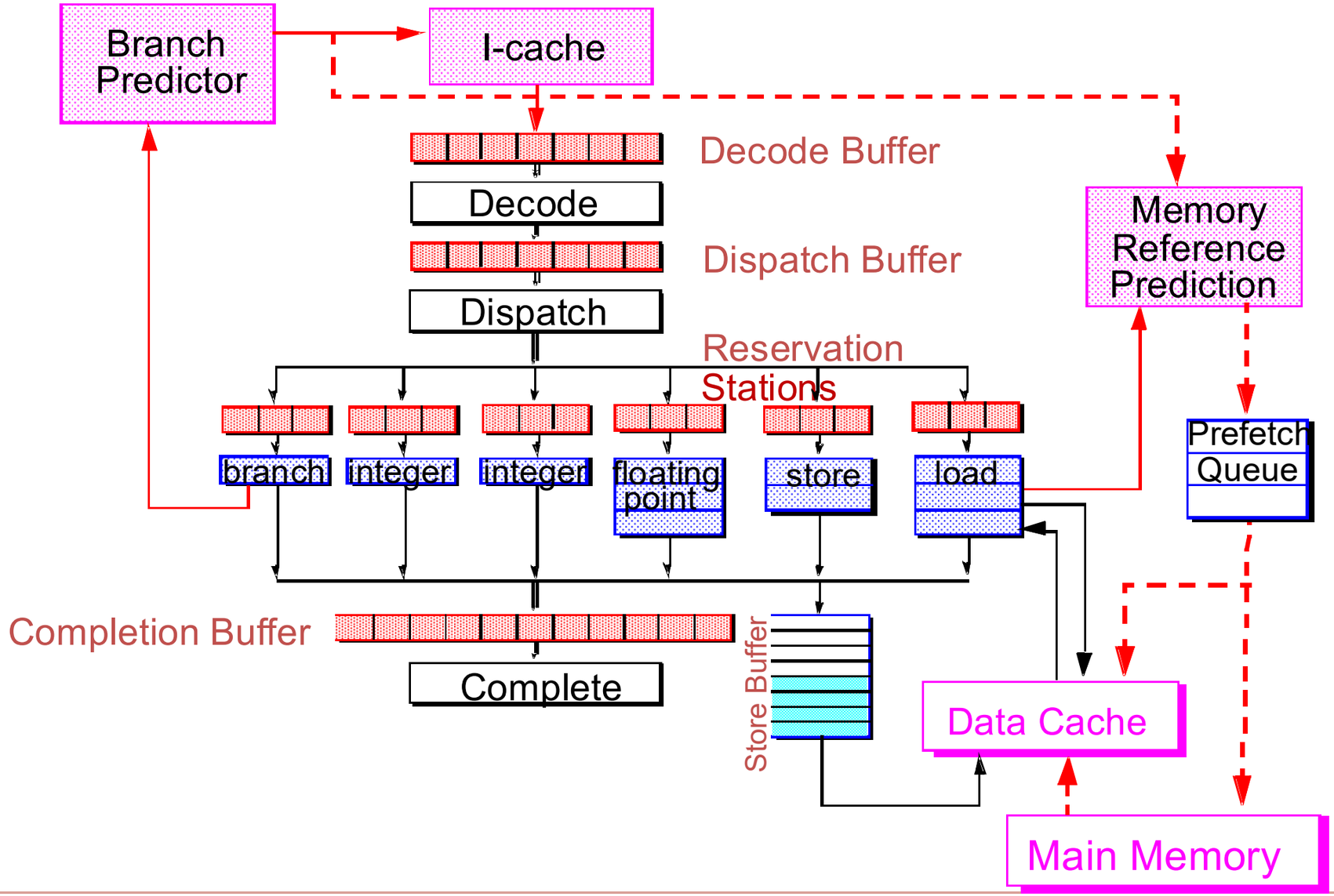
# Dual-Ported Non-Blocking Cache



# Non-blocking or Lockup Free Caches

- Idea:
  - Allow for hits while serving a miss (hit-under-miss)
  - Allow for more than one outstanding miss (miss-under-miss)
- When does it make sense (for L1, L2, …)
  - When the processor can handle >1 pending load/store
    - This is the case with superscalar processors
  - When the cache serves >1 processor or other cache
  - When the lower level allows for multiple pending accesses
    - Multi-banked, split transaction busses, pipelining, …
- What is difficult about non-blocking caches:
  - Handling multiple misses at the time
  - Handling loads to pending misses
  - Handling stores to pending misses

# Prefetching Data Cache



# 18-740: Modern Computer Architecture & Design

## Lecture 8:
## "Modern Memory Dataflow and Data Storage"

A. Long Latency Memory Instructions

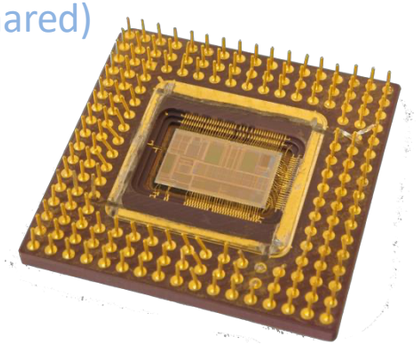
B. Why Should We Care About Memory (MLP)

C. Modern Memory Hierarchy and Data Storage (DRAM)

Electrical & Computer ENGINEERING

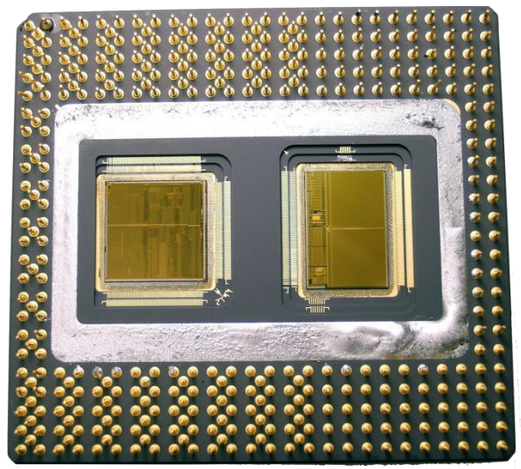
# "Urban Sprawl": off-chip to on-chip Memories

**Intel i486** – 8K on-chip L1
(Shared)



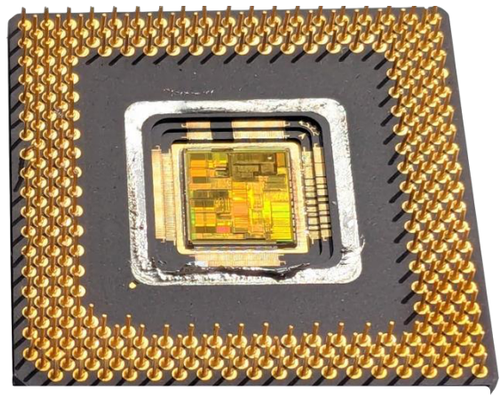
https://commons.wikimedia.org/wiki/File:Intel_i486_DX.jpg#filelinks

**Intel P6** – 16K on-chip L1
(8k Data 8k Instruction)
256KB on-package L2
(Shared)



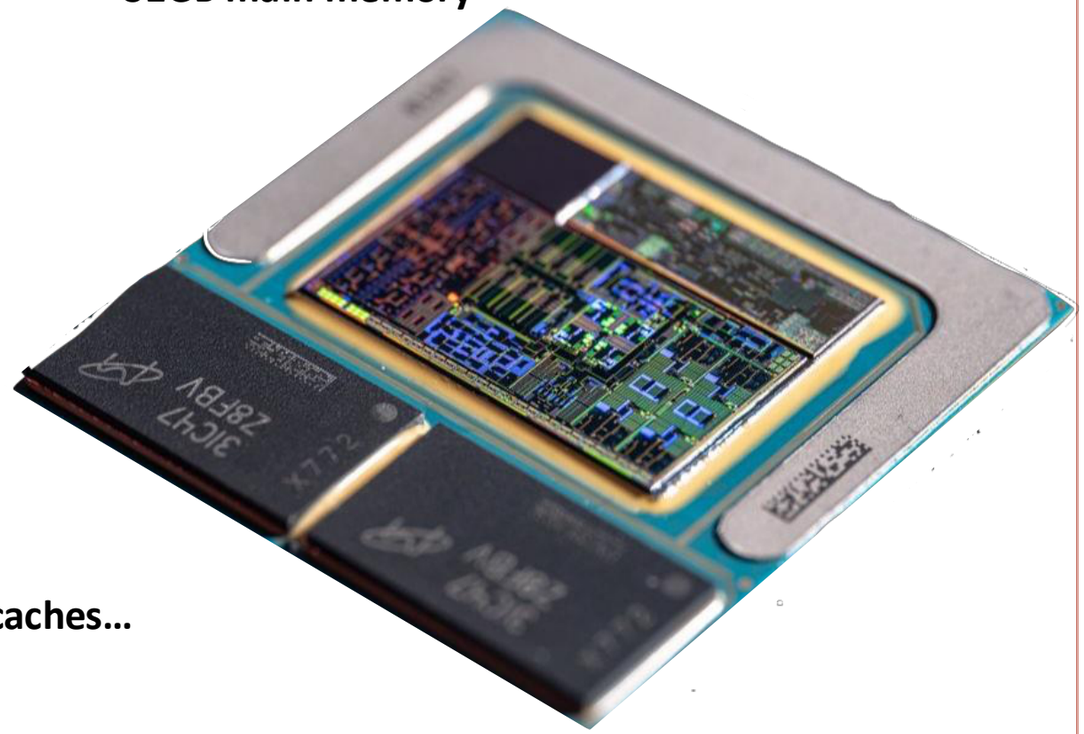
By Moshen - http://en.wikipedia.org/wiki/Image:Pentiumpro_moshen.jpg, CC BY-SA 2.5, https://commons.wikimedia.org/w/index.php?curid=2262471

**Intel P5** – 16K on-chip L1
(8k Data 8k Instruction)



By Ken Shirriff - https://static.righto.com/images/pentium-rug/pentiums.jpg

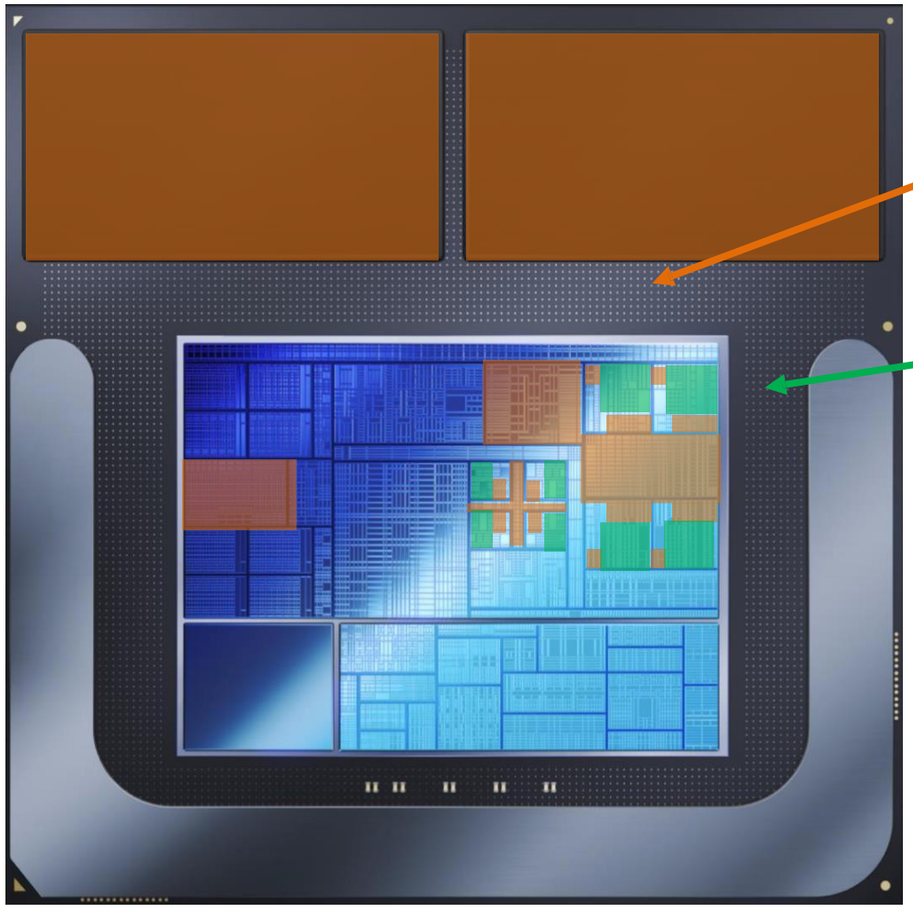
**Intel Lunar Lake** –
48KB/64KB p-core L0D/I, 192KB p-core L1D
3MB p-core L2D, 12MB p-core shared L3
32KB/64KB e-core L1D/I, 4MB e-core shared L2
8MB GPU L2 cache, 8MB memory side cache,
32GB **main memory**



By Fritzchen Fritz - https://www.tomshardware.com/pc-components/cpus/intels-lunar-lake-intricacies-revealed-in-new-hi-resolution-die-shots

**Plus TLBs, Trace caches, µOP caches…**

# “Urban Sprawl”: off-chip to on-chip Memories



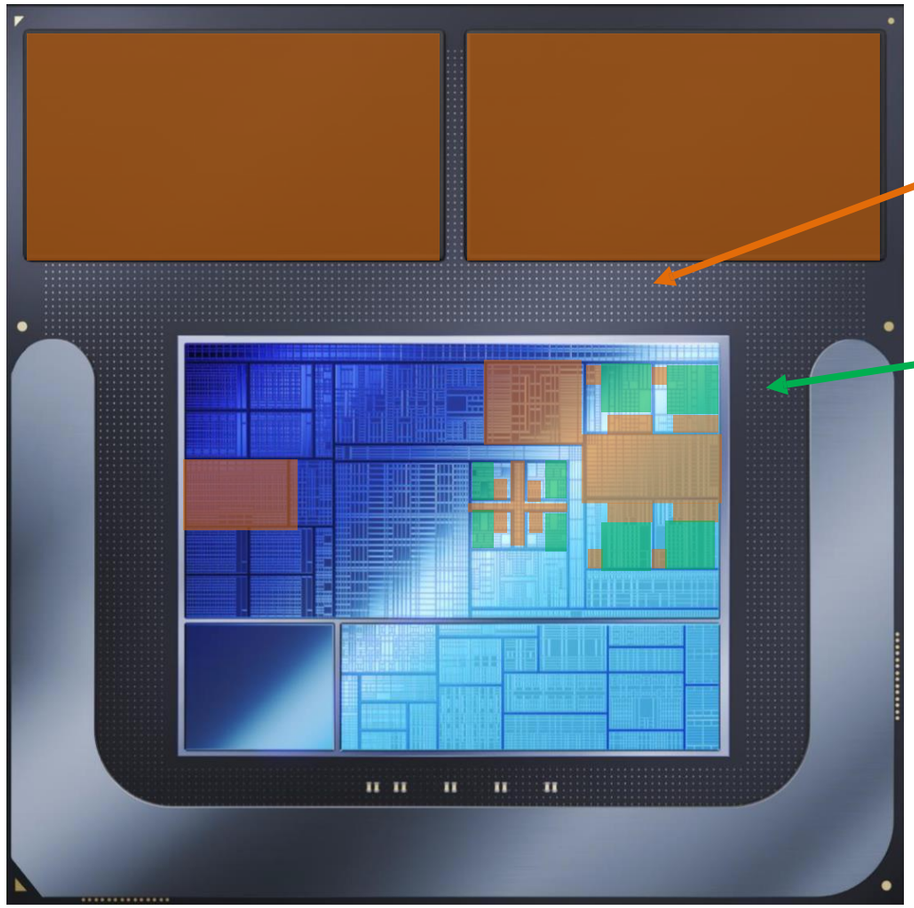
Main and Cache Memories

CPU compute

HotChips 2024, Arik Gihon, Intel Lunar Lake Architecture Session

Placement of boxes on die is not intended to be accurate, but takes publicly available information to attempt to convey how much area on package is dedicated to memory versus compute

# “Urban Sprawl”: off-chip to on-chip Memories



HotChips 2024, Arik Gihon, Intel Lunar Lake Architecture Session

Placement of boxes on die is not intended to be accurate, but takes publicly available information to attempt to convey how much area on package is dedicated to memory versus compute

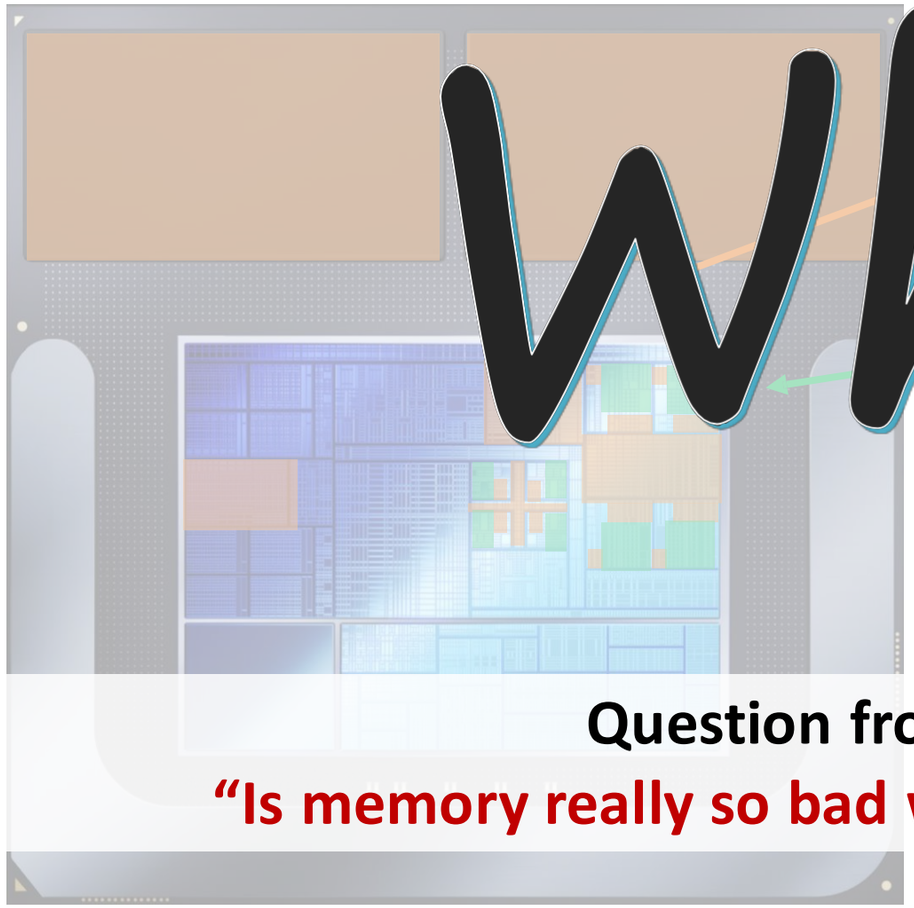
Main and Cache Memories

CPU compute

1. SoC package is pulling in all the memory (and compute) that used to be external on the motherboard.

2. Designers are biting the immense (area, power, coherence) cost of data duplication across inclusive layers.

# "Urban Sprawl": off-chip to on-chip Memories



HotChips 2024, Arik Gihon, Intel Lunar Lake Architecture Session

Placement of boxes on die is not intended to be accurate, but takes publicly available information to attempt to convey how much area on package is dedicated to memory versus compute

Why?

Main and Cache Memories

CPU compute

**1.** SoC package is pulling in all the memory (and compute) that used to be external on the motherboard.

**2.** Designers are biting the immense (area, power, coherence) cost of data duplication across inclusive layers.

**Question from 18-447 student:**

**"Is memory really so bad we have to duplicate it 9 times?"**

# Roofline Performance Model

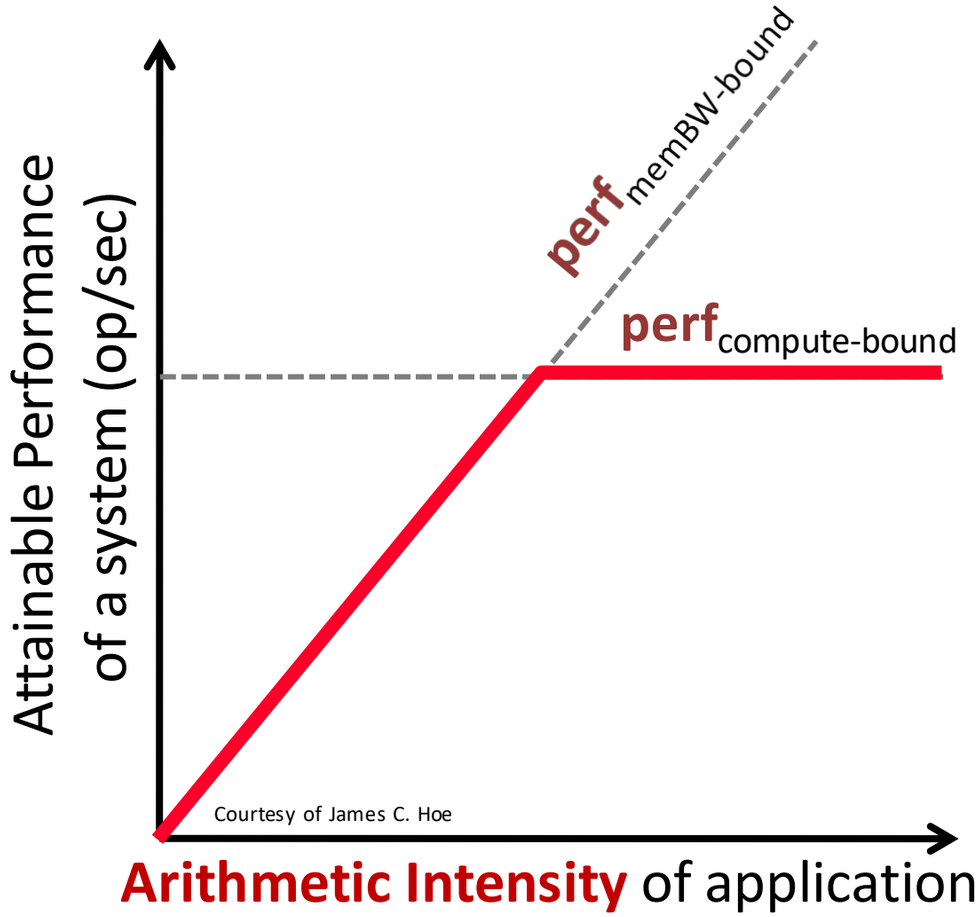
[Williams & Patterson, 2006]

- The **Arithmetic Intensity** is how much productive work is "unlocked" per Byte fetched from memory

I.e. some avg of the load dependency chain lengths

```
Load R1 <- Mem[R3]
Add R2 <- R1 + 25
Add R5 <- R0 + 6
Fmul F3 <- R2 * R1
Fmul F4 <- F3 * 0.3
Add R5 <- R0 + 6
Cmp F3 > F4 <label>
// … and so on
```

1, 2, 3, 4

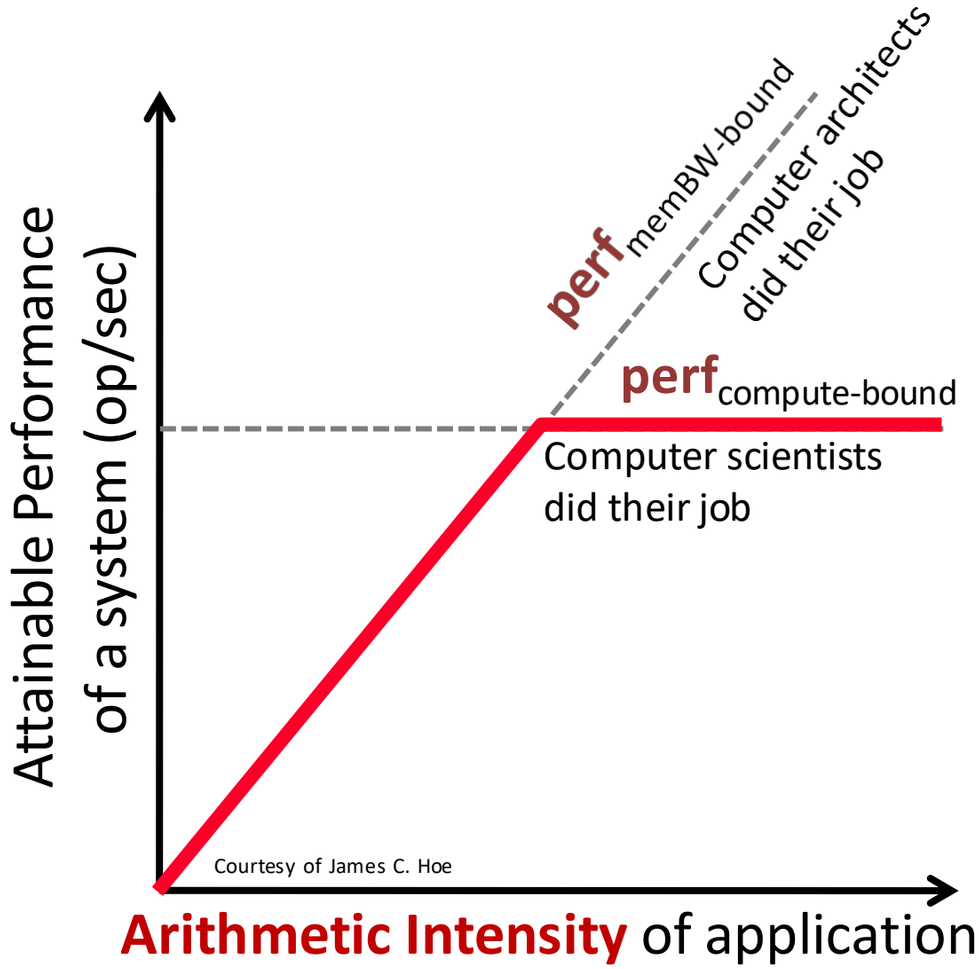


# Roofline Performance Model

[Williams & Patterson, 2006]

- A **Compute-Bound** workload
  - Unhidden latency is mostly compute
  - Rarely true for general-purpose RISC CPU code
- A **memBW-Bound** workload
  - Unhidden latency is mostly memBW saturation
  - I.e. Latency = 1 / peak (saturated) memory BW
- What about **memLatency-Bound**?
  - Roofline model assumes you can saturate memory
  - I.e. workload has enough MLP,
  - and the processor can support it.

What if unhidden latency is just… memory latency?



# Meet the Memory Gap

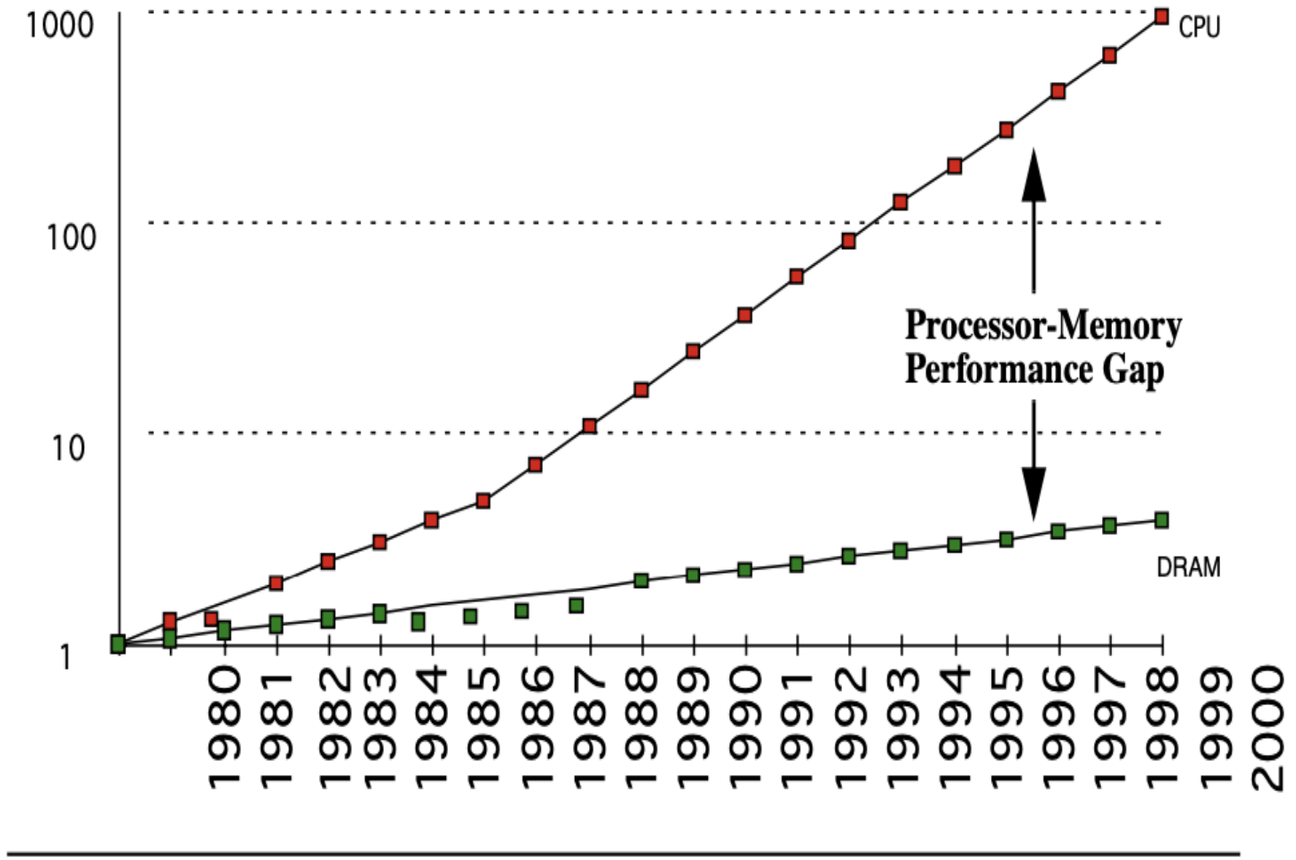



FIGURE 1. Processor-Memory Performance Gap.[Hen96].

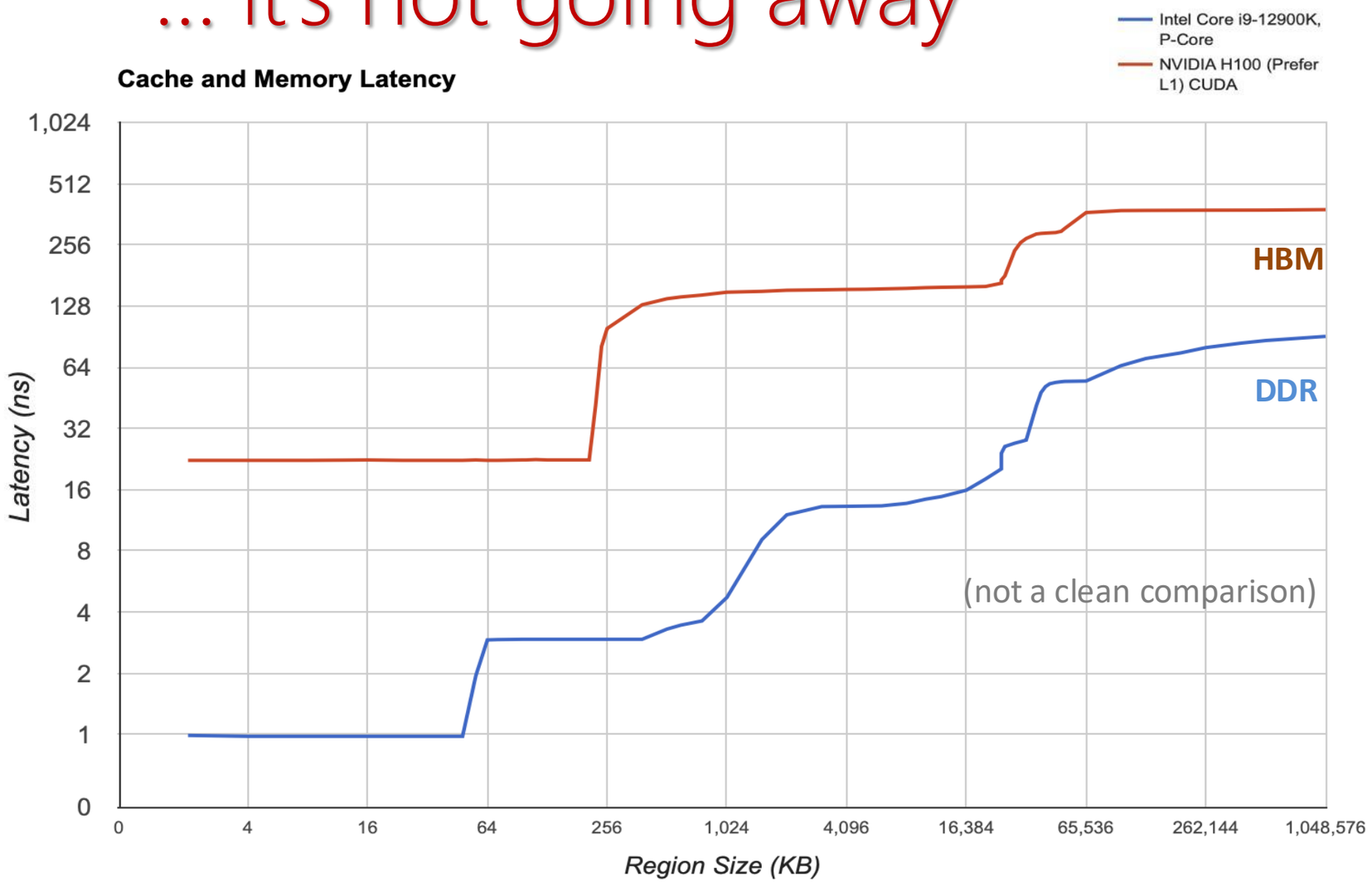
| Processor | Alpha 21164 | |
|---|---|---|
| Machine | AlphaServer 8200 | |
| Clock Rate | 300 MHz | |
| Memory Performance | Latency | Bandwidth |
| I Cache (8KB on chip) | 6.7 ns (2 clocks) | 4800 MB/sec |
| D Cache (8KB on chip) | 6.7 ns (2 clocks) | 4800 MB/sec |
| L2 Cache (96KB on chip) | 20 ns (6 clocks) | 4800 MB/sec |
| L3 Cache (4MB off chip) | 26 ns (8 clocks) | 960 MB/sec |
| Main Memory Subsystem | 253 ns (76 clocks) | 1200 MB/sec |
| Single DRAM component | ≈60ns (18 clocks) | ≈30–100 MB/sec |

| Category | SPECint92 | SPECfp92 | Database | Sparse |
|---|---|---|---|---|
| Clocks Per Instruction (CPI) | 1.2 | 1.2 | 3.6 | 3.0 |
| I cache misses per 1000 instructions | 7 | 2 | 97 | 0 |
| D cache misses per 1000 instructions | 25 | 47 | 82 | 38 |
| L2 cache misses per 1000 instructions | 11 | 12 | 119 | 36 |
| L3 cache misses per 1000 instructions | 0 | 0 | 13 | 23 |
| Fraction of time in processor | 0.78 | 0.68 | 0.23 | 0.27 |
| Fraction of time in I cache misses | 0.03 | 0.01 | 0.16 | 0.00 |
| Fraction of time in D cache misses | 0.13 | 0.23 | 0.14 | 0.08 |
| Fraction of time in L2 cache misses | 0.05 | 0.06 | 0.20 | 0.07 |
| Fraction of time in L3 cache misses | 0.00 | 0.02 | 0.27 | 0.58 |

**Failure to use excess bandwidth to hide latency**

[1] “A Case for Intelligent RAM: IRAM”, Patterson et al, IEEE Micro 1997

# … it's not going away




Generally (and for good reason)
Recent focus is on

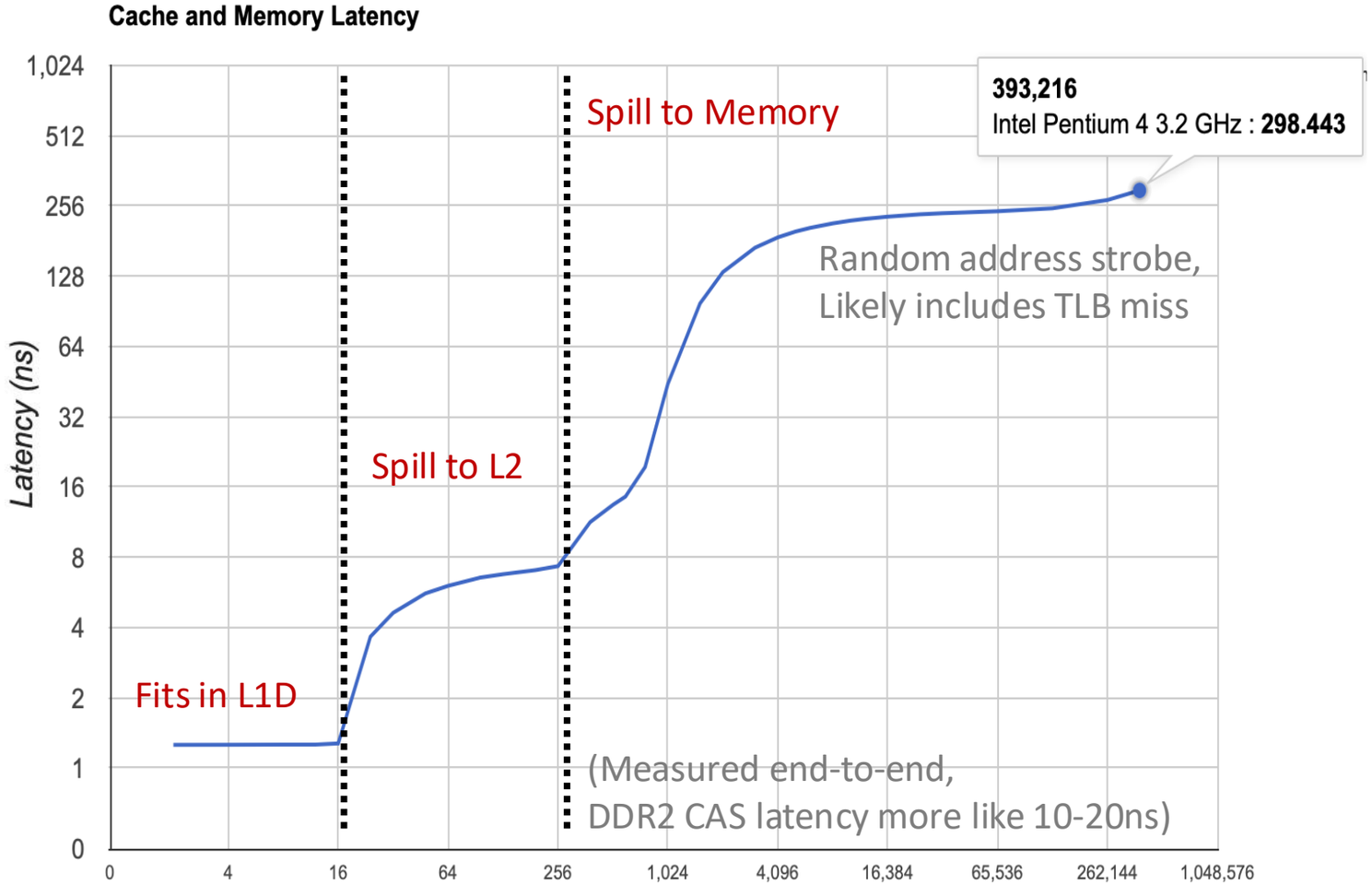
**High-Bandwidth Memory**
More than
**Low-Latency Memory**

(You can convert bandwidth into latency by exploiting parallelism,

but there is less of this to work with on a CPU than a GPU)

Data from Chips and Cheese Memory Latency Tests (https://jsmemtest.chipsandcheese.com/latencydata)

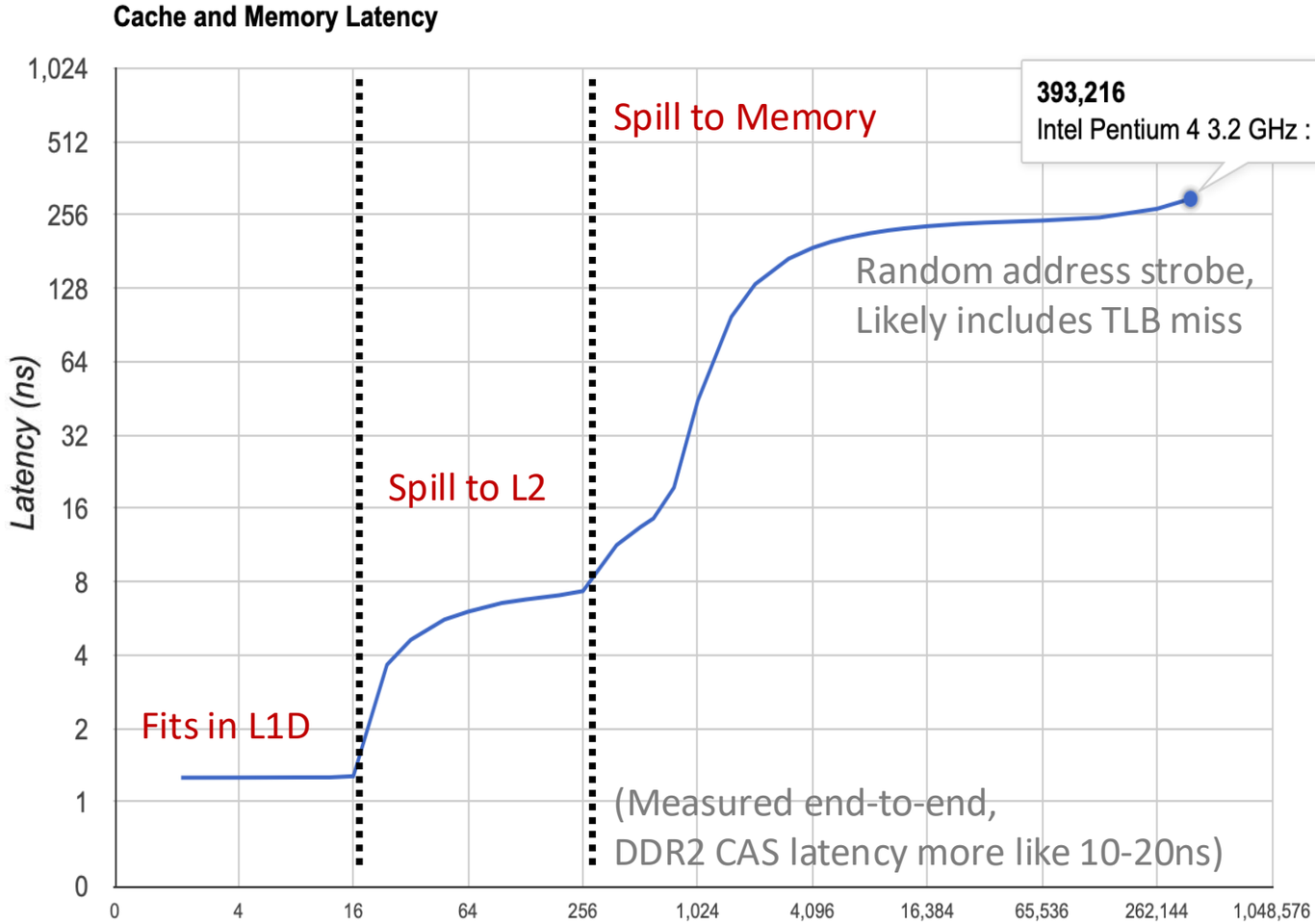
# Pentium 4 (system) memory latency ~300ns




Data from Chips and Cheese Memory Latency Tests (https://jsmemtest.chipsandcheese.com/latencydata)

[1] “The Microarchitecture of the Pentium 4 Processor”, Hinton et al, Intel technology journal 2001

# Napkin math on how much you should care




3.2 ***GHz*** = 1 ***CPU cycle*** / 0.3125 ***ns***

300 ***ns*** / 0.3125 ***ns*** = 960 ***CPU cycles***

Putting that in terms of arithmetic instructions…

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| TC Nxt IP | | TC Fetch | | Drive | Alloc | Rename | | Que | Sch | Sch | Sch | Disp | Disp | RF | RF | Ex | Flgs | Br Ck | Drive |

960 / 20 = 48 ***end-to-end arithmetic instructions***

Given P4 can retire 3 µOps per cycle…

960 * 3 = 2880 ***µOps could have been completed***

(running at full tilt, which is very unlikely)

**ROB is only ~126 entries**

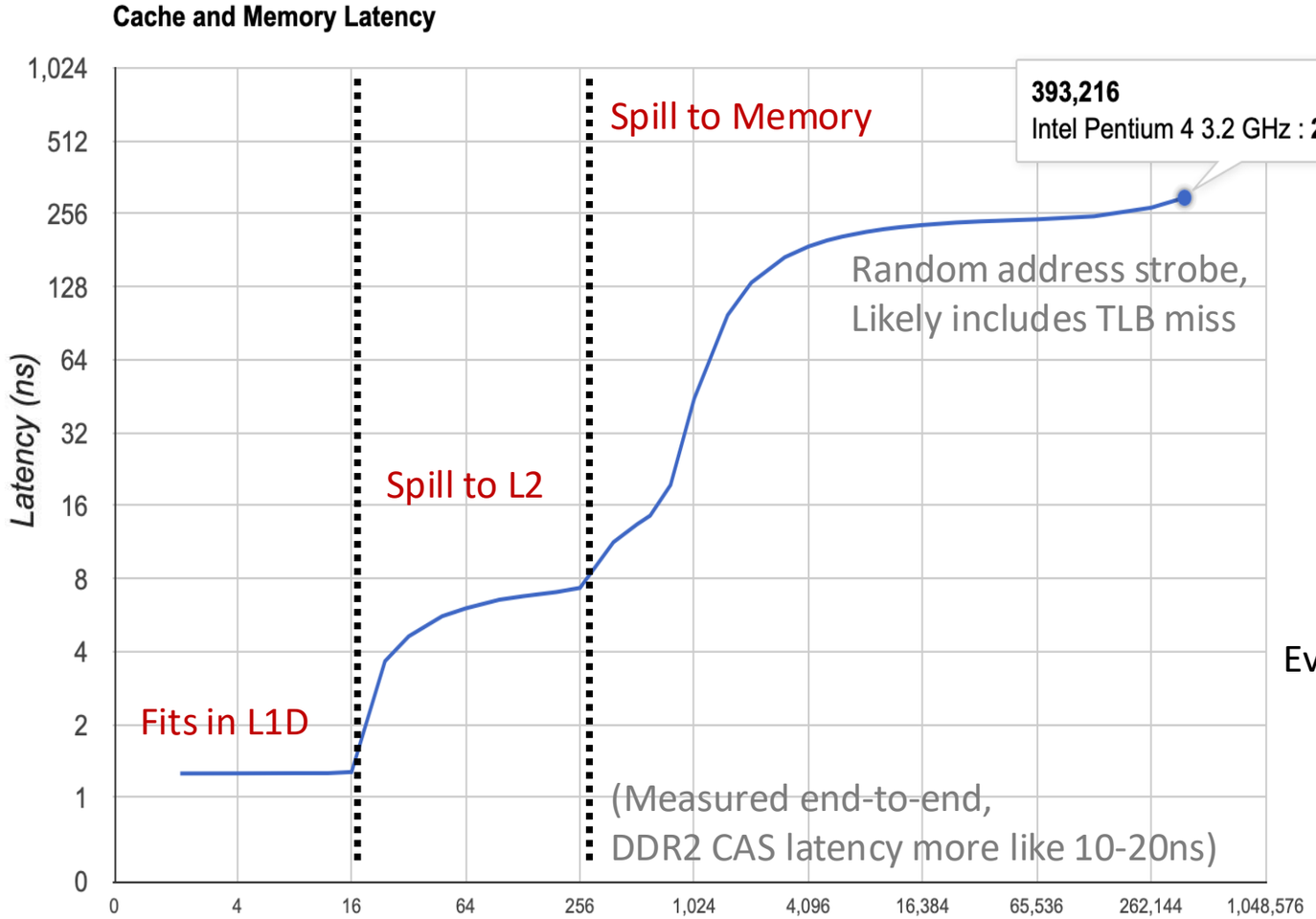
This could stall the pipeline…

**by how much?**

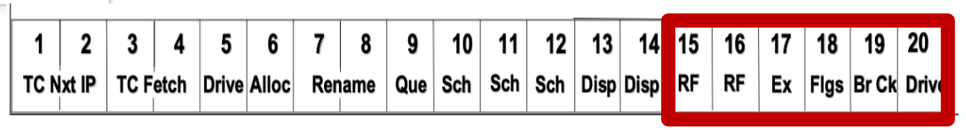
Data from Chips and Cheese Memory Latency Tests (https://jsmemtest.chipsandcheese.com/latencydata)

[1] “The Microarchitecture of the Pentium 4 Processor”, Hinton et al, Intel technology journal 2001

# Napkin math on how much you should care




Assuming 125 *older* integer µOps in RS




They all wait until the 126th µOp (the load) misses

P4 has two integer pipes

6 cycles until the first two retire

+

2 ready for retirement every cycle after

Covers ~**66 cycles**

Even if it can execute another 125 *younger* arithmetic µOps
(can't dispatch any more since retirement is stalled)
(ignoring the latency impact of finite-width retirement)

Approximately another ~**60 cycles**

**Still ~800 cycles unhidden latency**

Data from Chips and Cheese Memory Latency Tests (https://jsmemtest.chipsandcheese.com/latencydata)

[1] "The Microarchitecture of the Pentium 4 Processor", Hinton et al, Intel technology journal 2001

# Latency hiding capacity of SS OOO CPU

// X is don't care – assume some reasonable high-ILP mix

// … ~125 arithmetic ops

Arith X <- X, X ~

Arith X <- X, X ~

Arith X <- X, X ~

Arith X <- X, X ~

Load X <- Mem[X] Arith X <- X, X

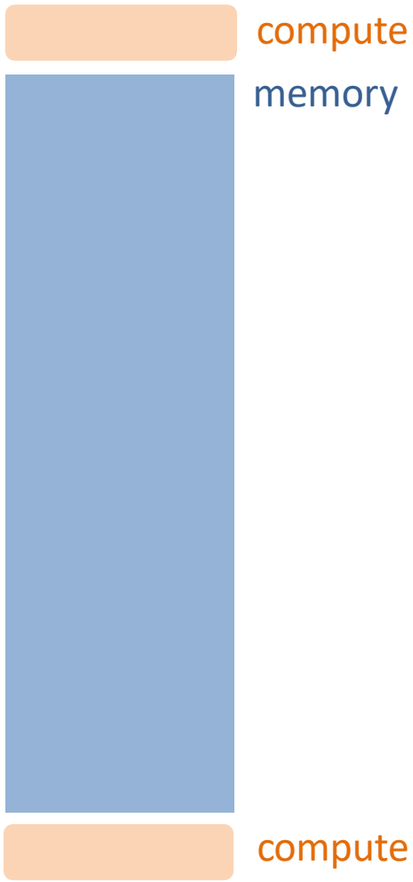
Arith X <- X, X

Arith X <- X, X

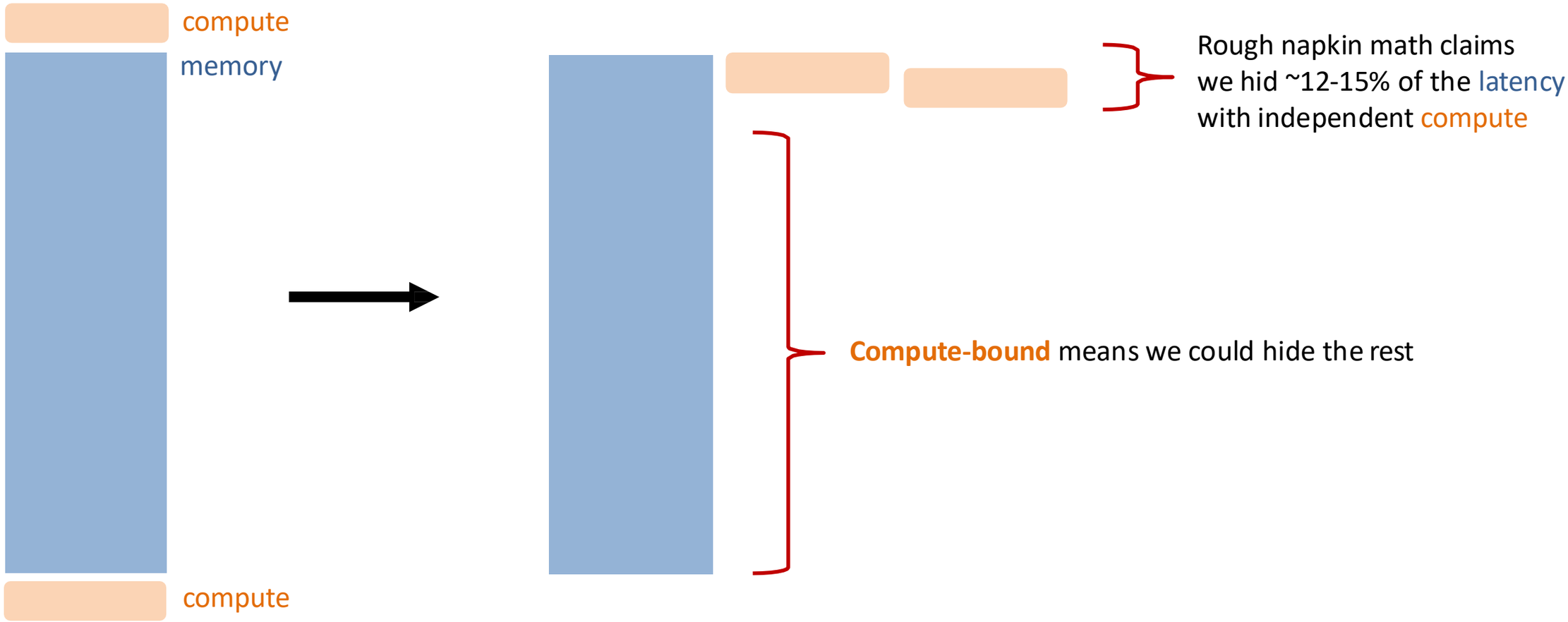
Arith X <- X, X

// … ~125 arithmetic ops

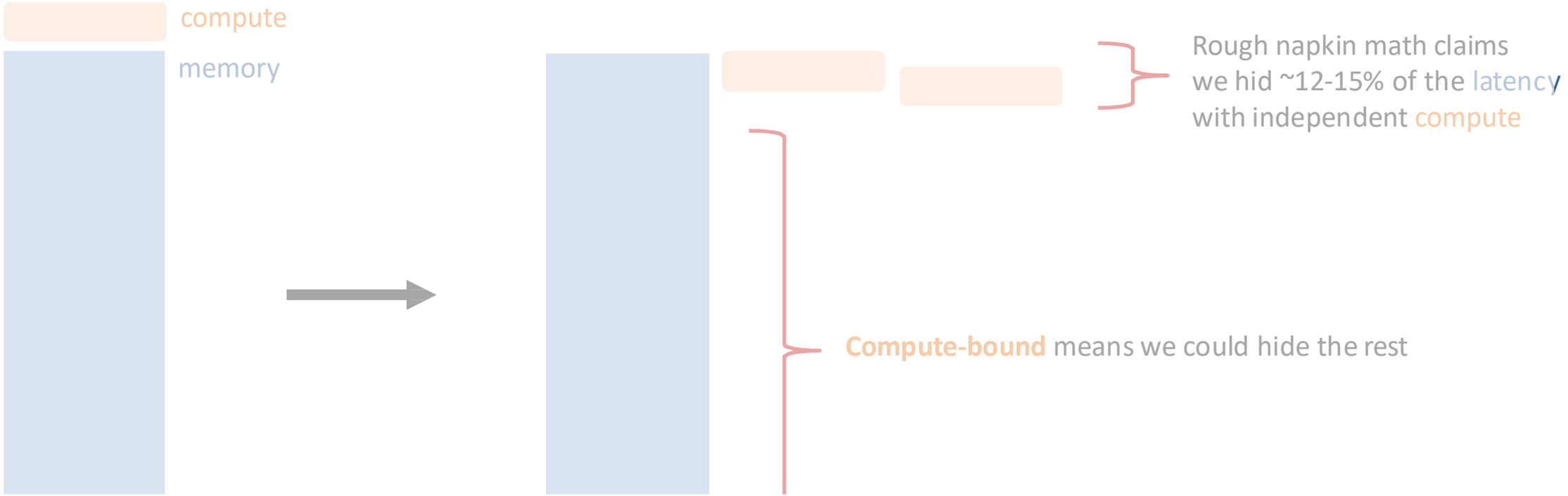
# Latency hiding capacity of SS OOO CPU



# Latency hiding capacity of SS OOO CPU



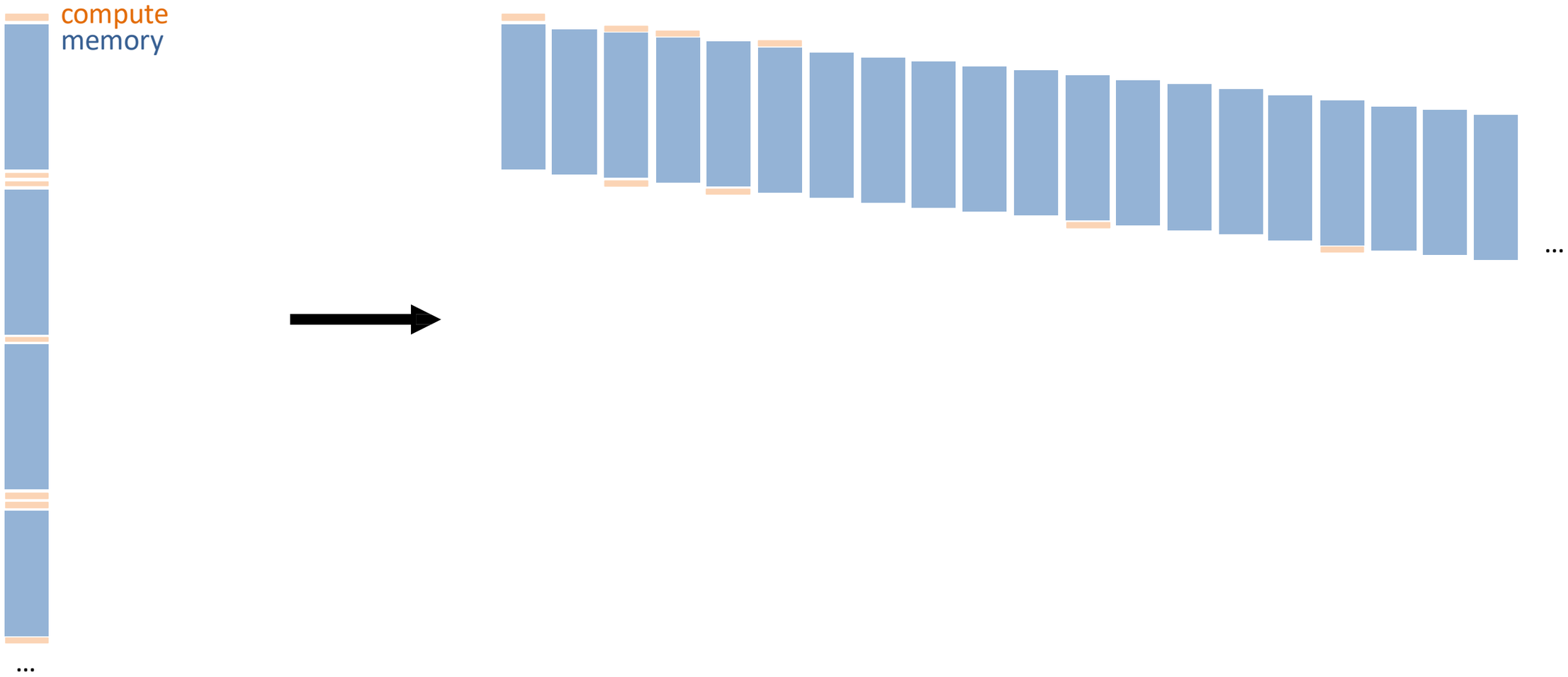
# Latency hiding capacity of SS OOO CPU



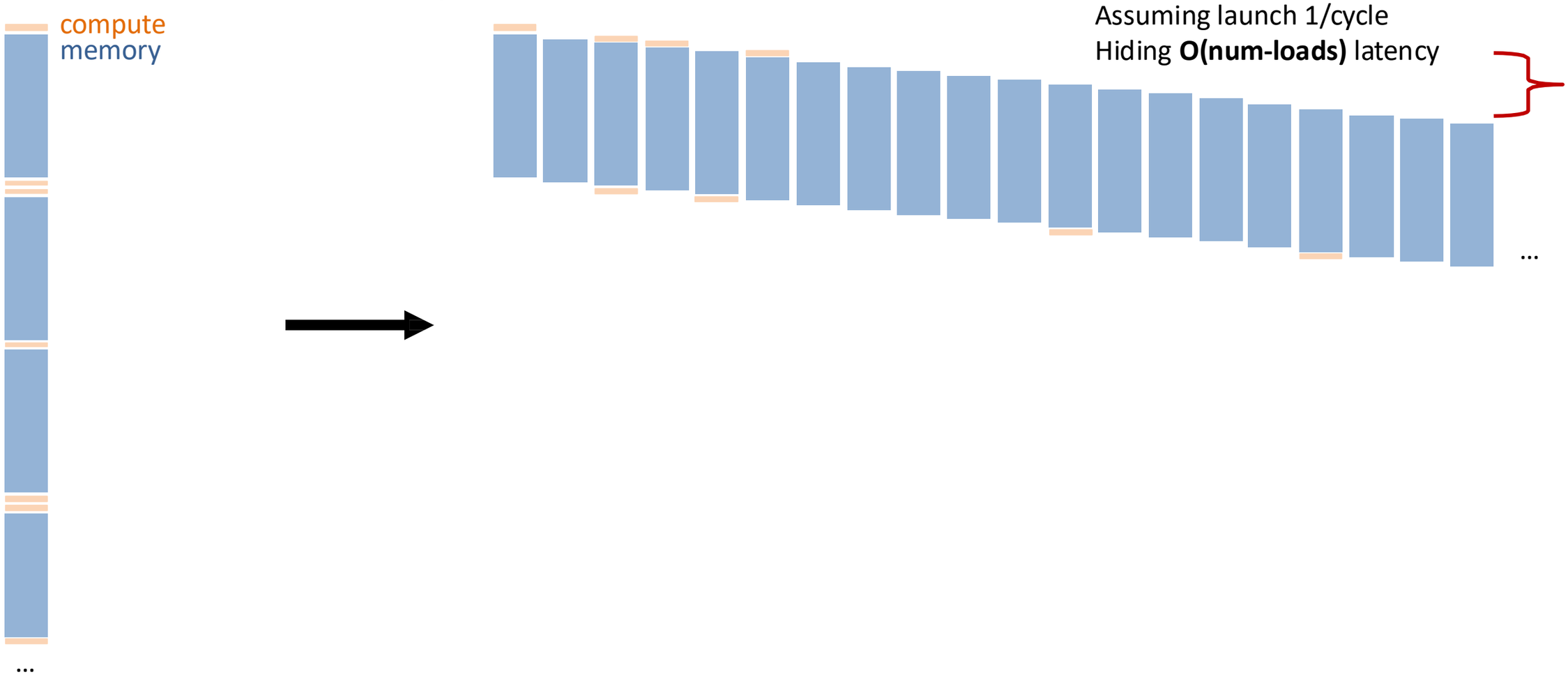

If memory operations are so slow,

Can we hide memory latency with **more memory latency**?

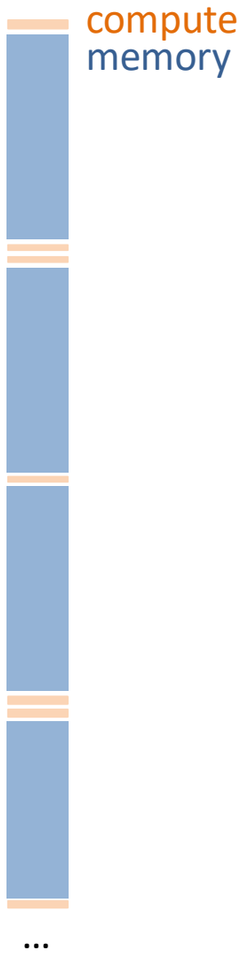
# CPU attempt to become BW-bound

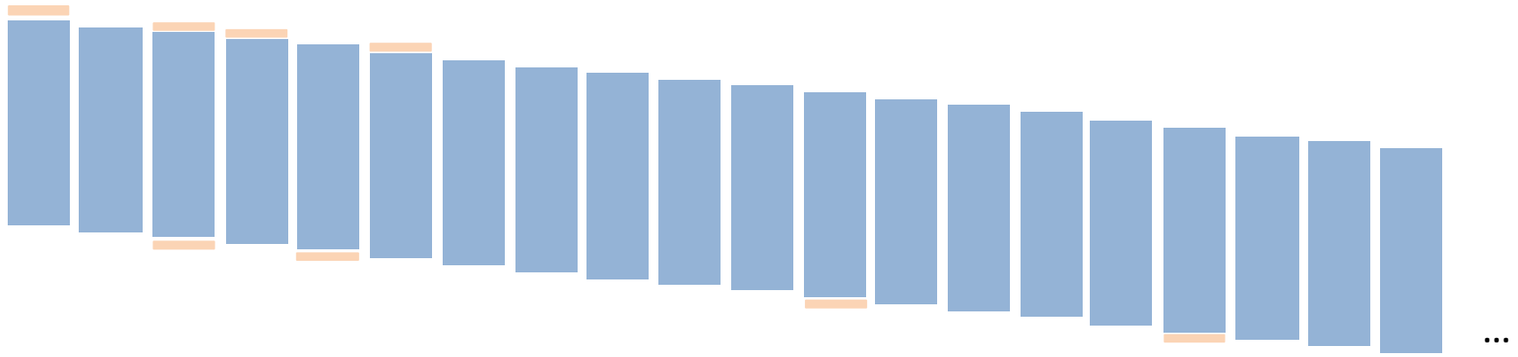


# CPU attempt to become BW-bound



# CPU attempt to become BW-bound



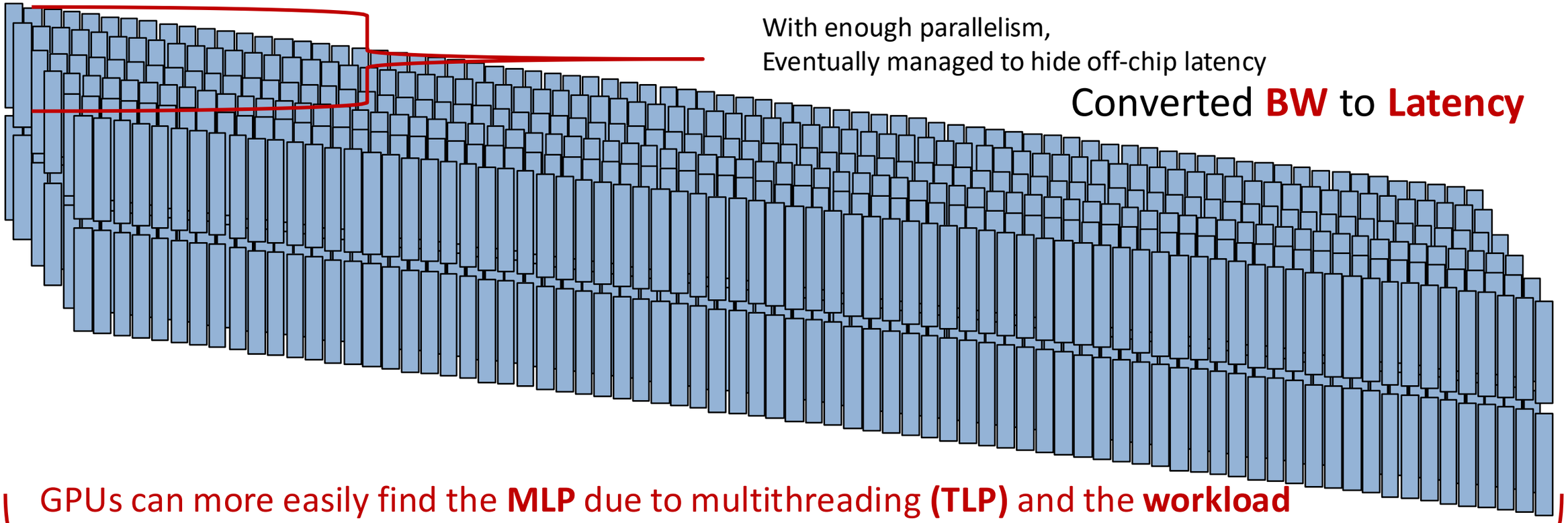

Can we even find enough **Memory-Level-Parallelism (MLP)** ?

P4 supports at most **48** outstanding loads
(i.e. load buffer size)

Assuming max rate is sending one miss per cycle,
Would need a lot more than that to hide **960** cycles



# GPU as a "Bandwidth machine"

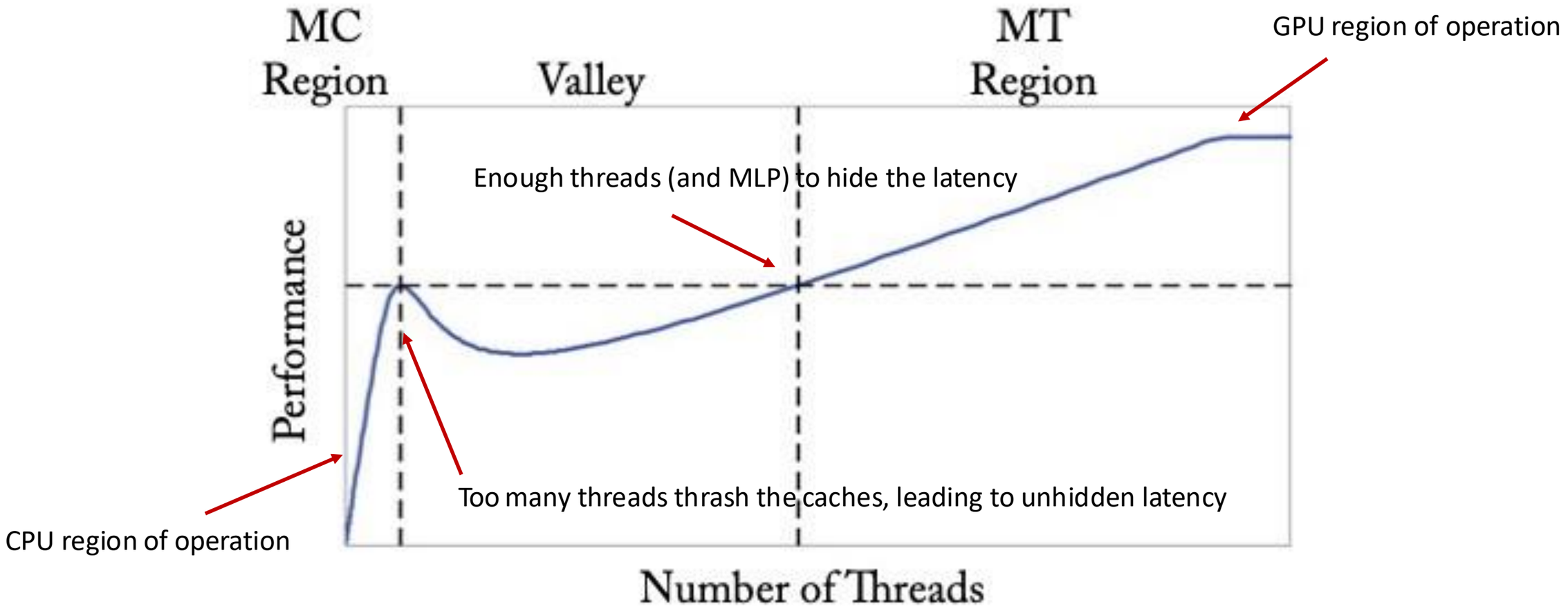



GPUs can more easily find the **MLP** due to multithreading **(TLP)** and the **workload**

Hence why GPU cache hierarchies are usually **shallower** – they can **generally hide more latency**

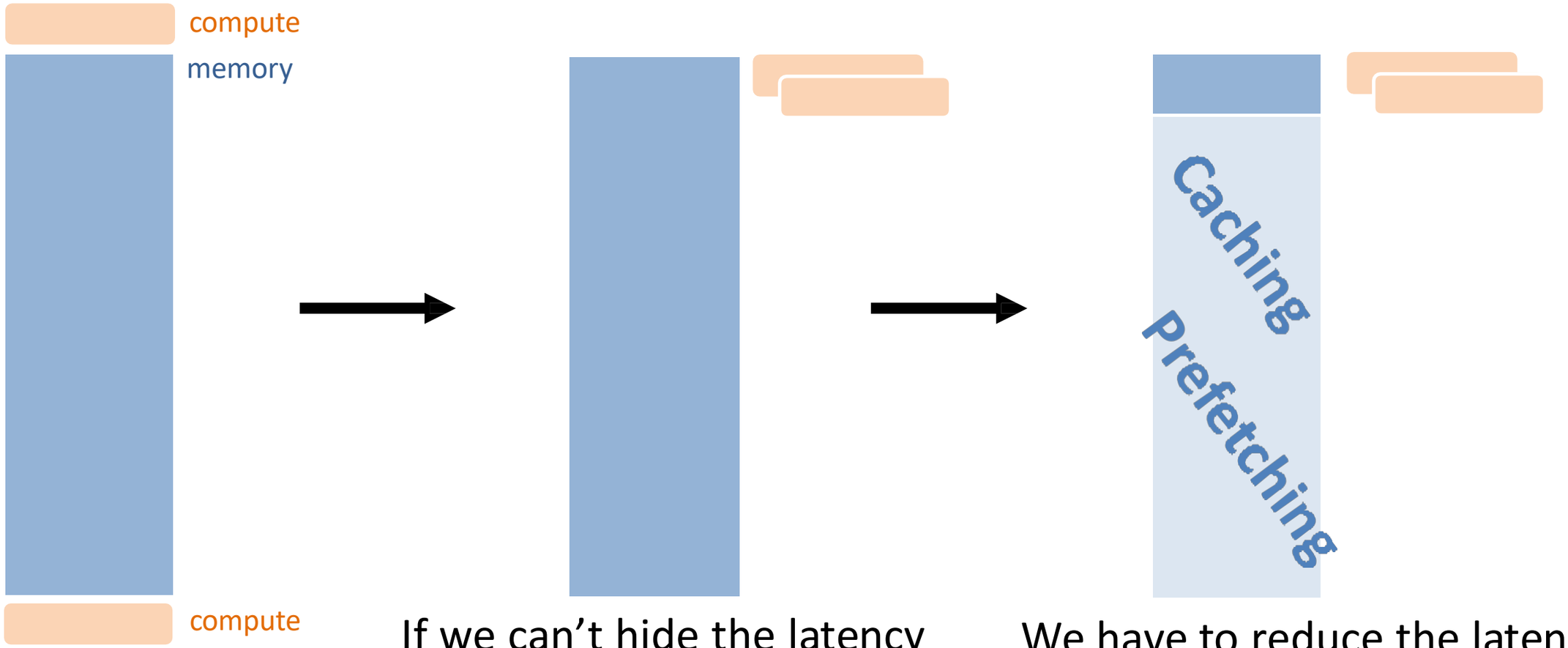
Relies on having enough actual RAM bandwidth to sustain… Hence tech like HBM

# GPU as a "Bandwidth machine"




[1] "General Purpose Graphics Architectures", Aamodt et al, Synthesis Lectures on Computer Architecture 2018

# CPU as a "Latency Machine"

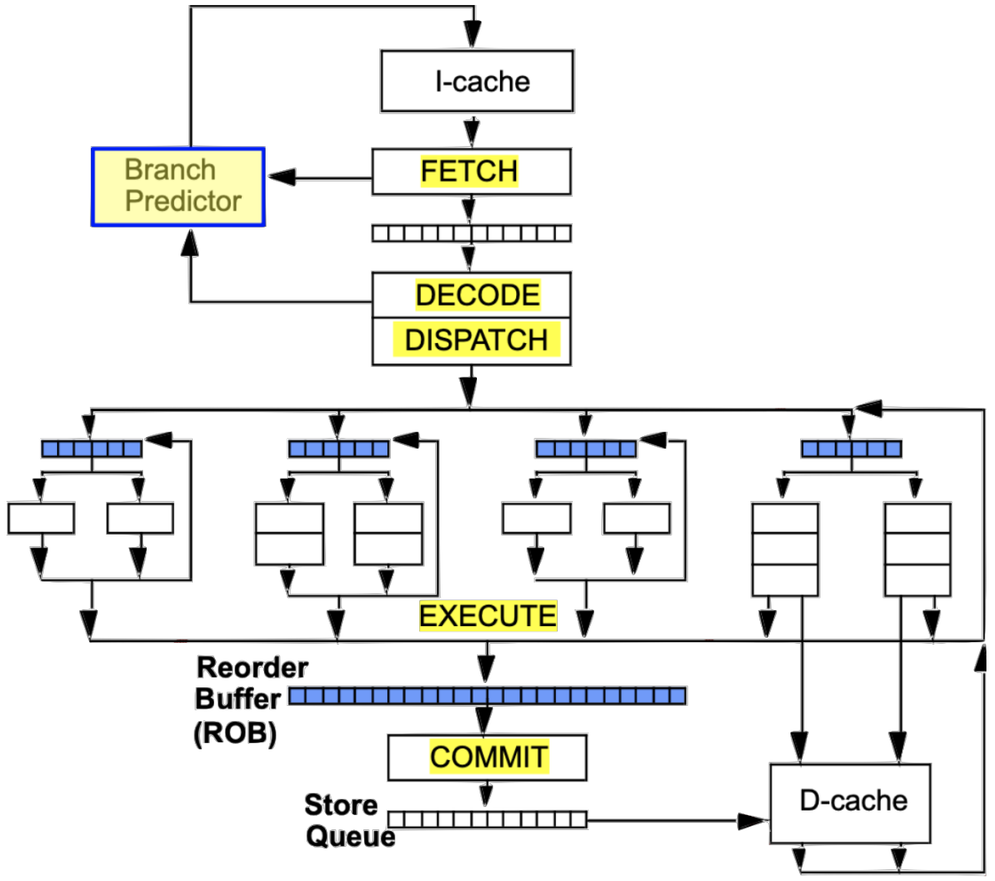



If we can't hide the latency

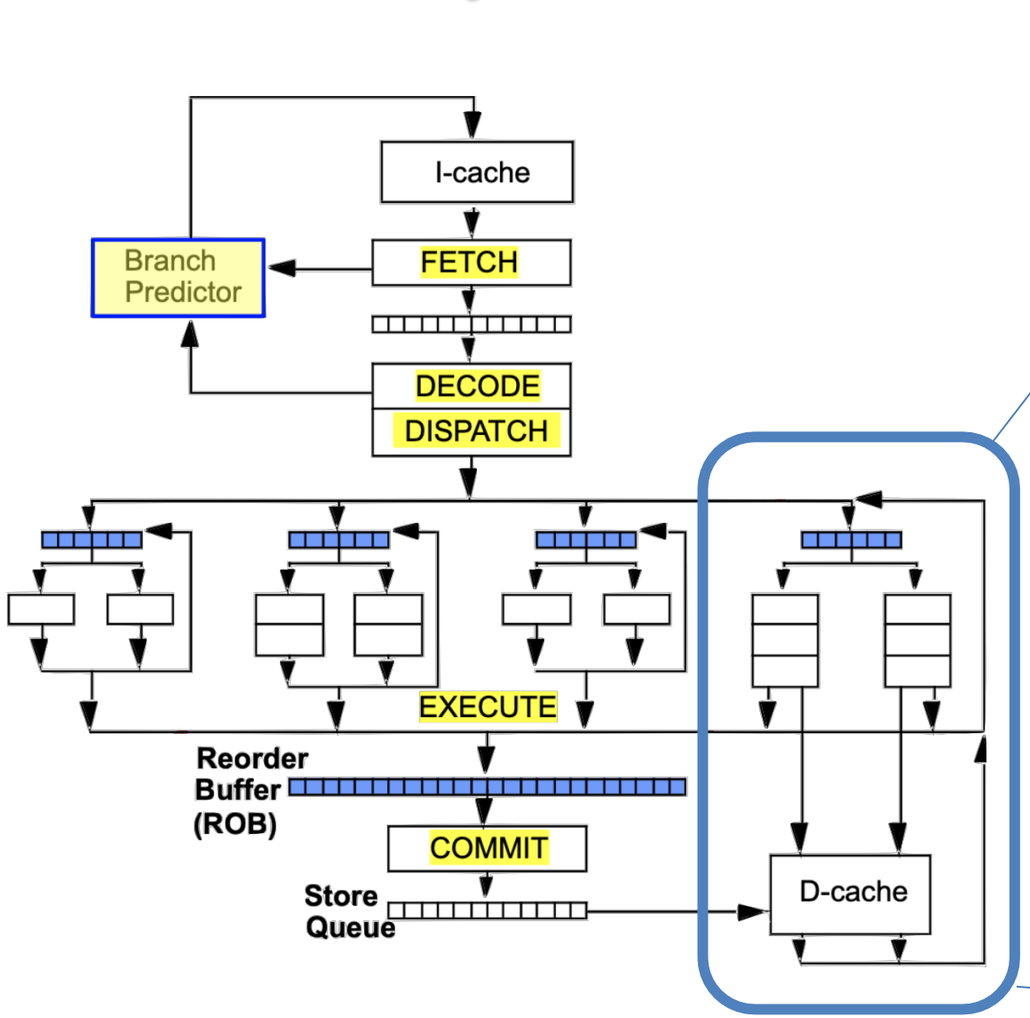
We have to reduce the latency

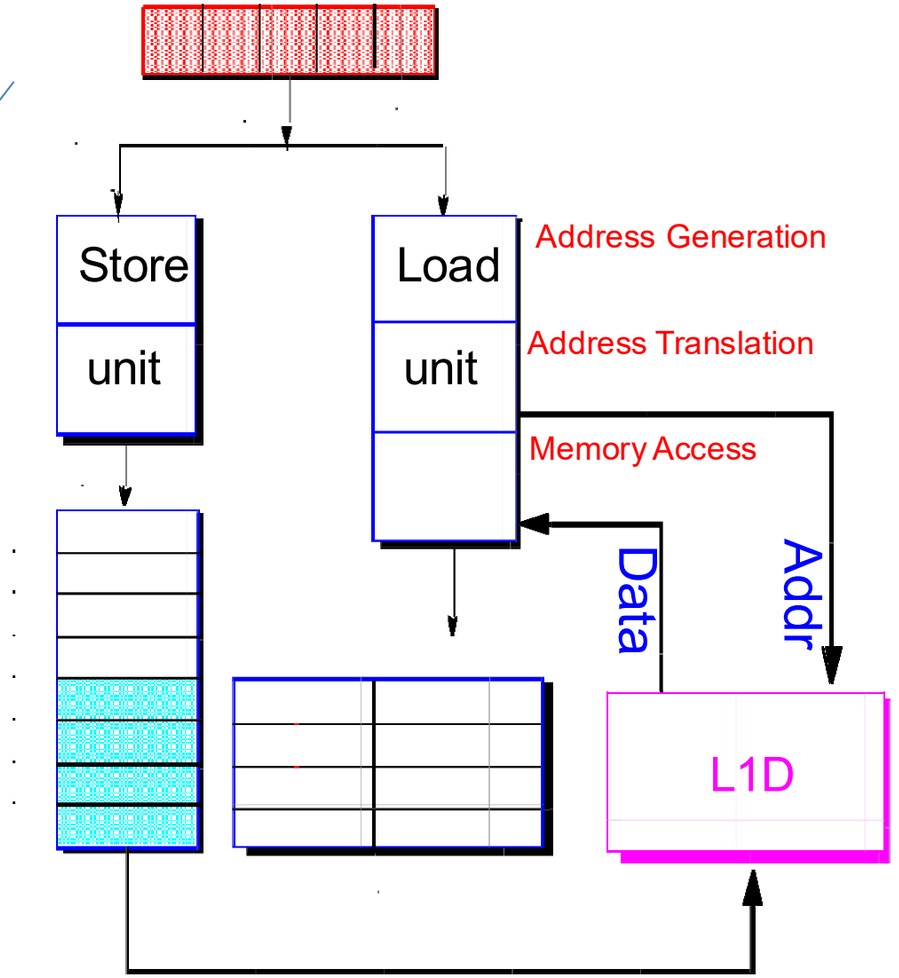
**Deeper cache hierarchies – smart replacement policies – high-coverage prefetchers**

# Memory Data Flow (so far)



# Memory Data Flow (so far)



# 18-740: Modern Computer Architecture & Design

## Lecture 8:
## "Modern Memory Dataflow and Data Storage"

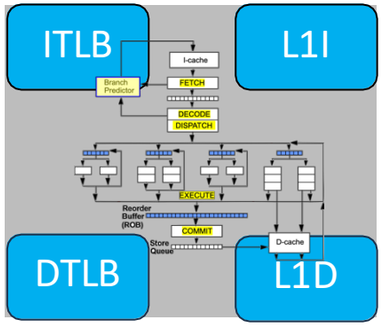
A. Long Latency Memory Instructions

B. Why Should We Care About Memory (MLP)

C. Modern Memory Hierarchy and Data Storage (DRAM)

Electrical & Computer ENGINEERING

# Memory Data Flow : Single Core



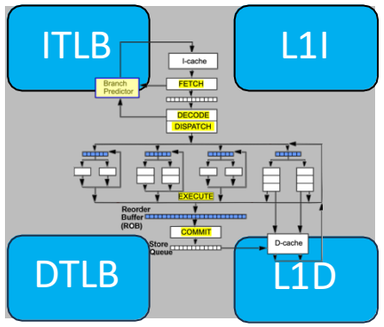

Unified Private Cache (SRAM)

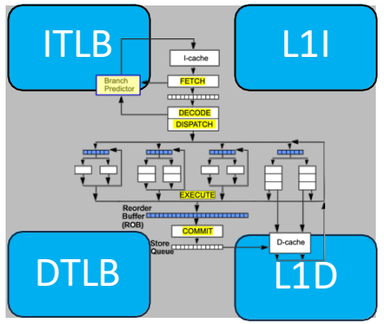
Unified Private Cache (SRAM)

Main Memory (DRAM)

Showing a genericized version of the Lunar Lake cluster hierarchy (split L1s + TLBs, unified L2 & LLC) – Ignoring the L0 as the 18740 processor does not have an L0

# Memory Data Flow : Multiple Core




Unified Private Cache (SRAM)

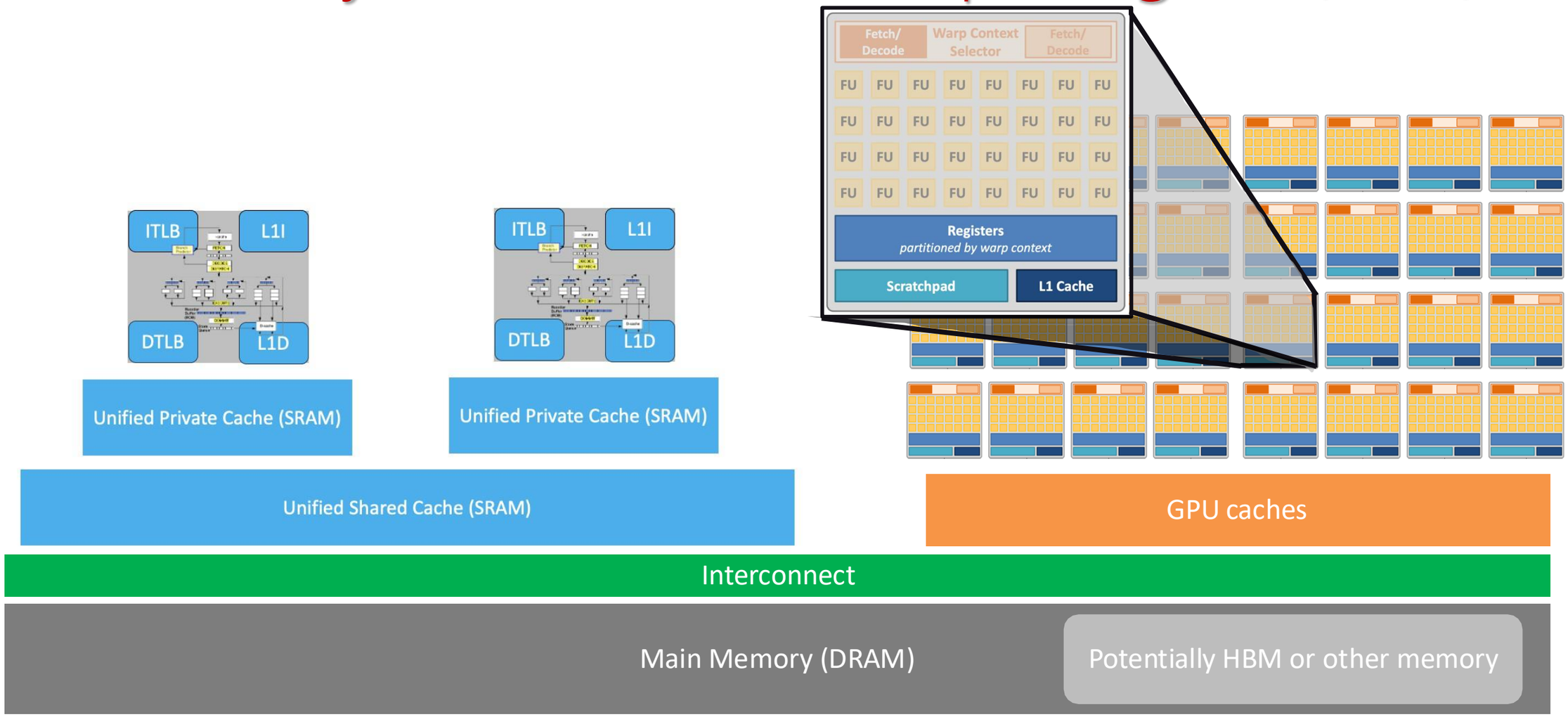
Unified Private Cache (SRAM)

Unified Shared Cache (SRAM)

Main Memory (DRAM)

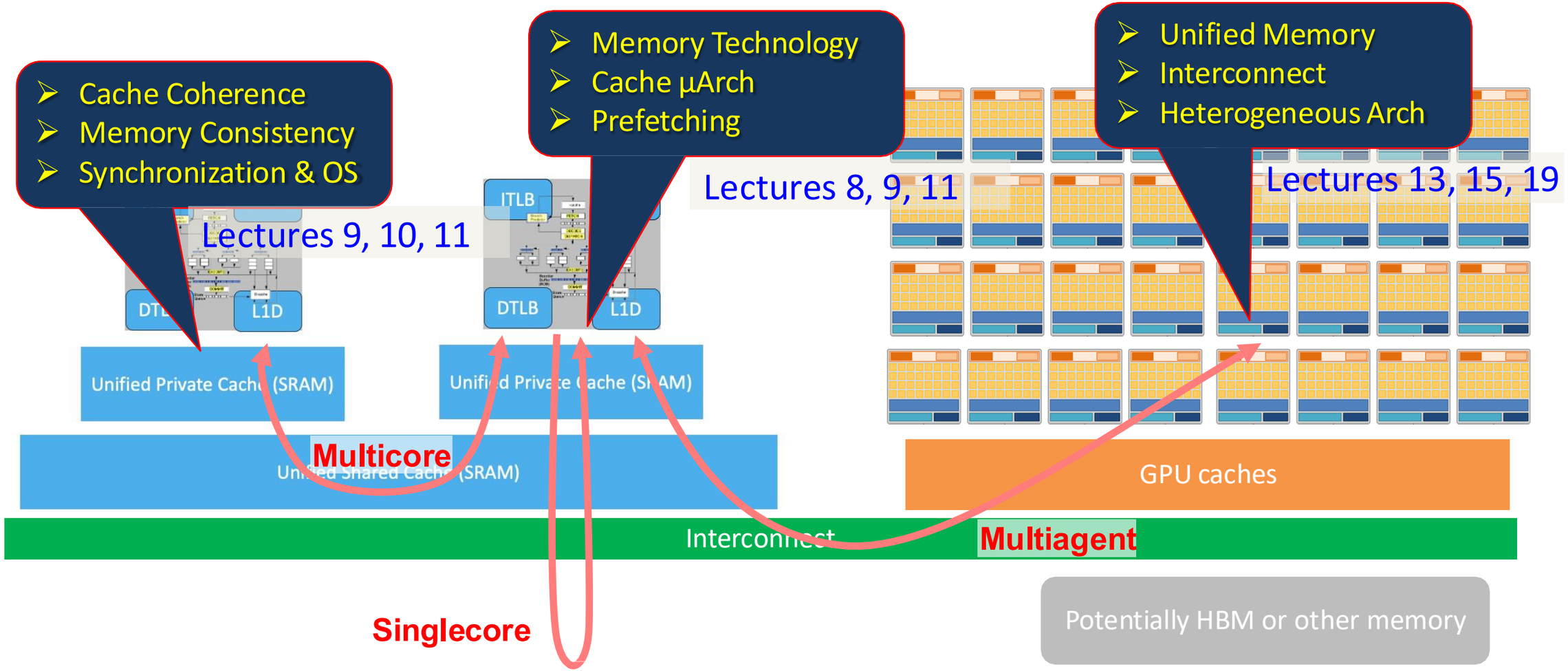
Showing a genericized version of the Lunar Lake cluster hierarchy (split L1s + TLBs, unified L2 & LLC) – Ignoring the L0 as the 18740 processor does not have an L0

# Memory Data Flow : Multiple Agent (SoC)



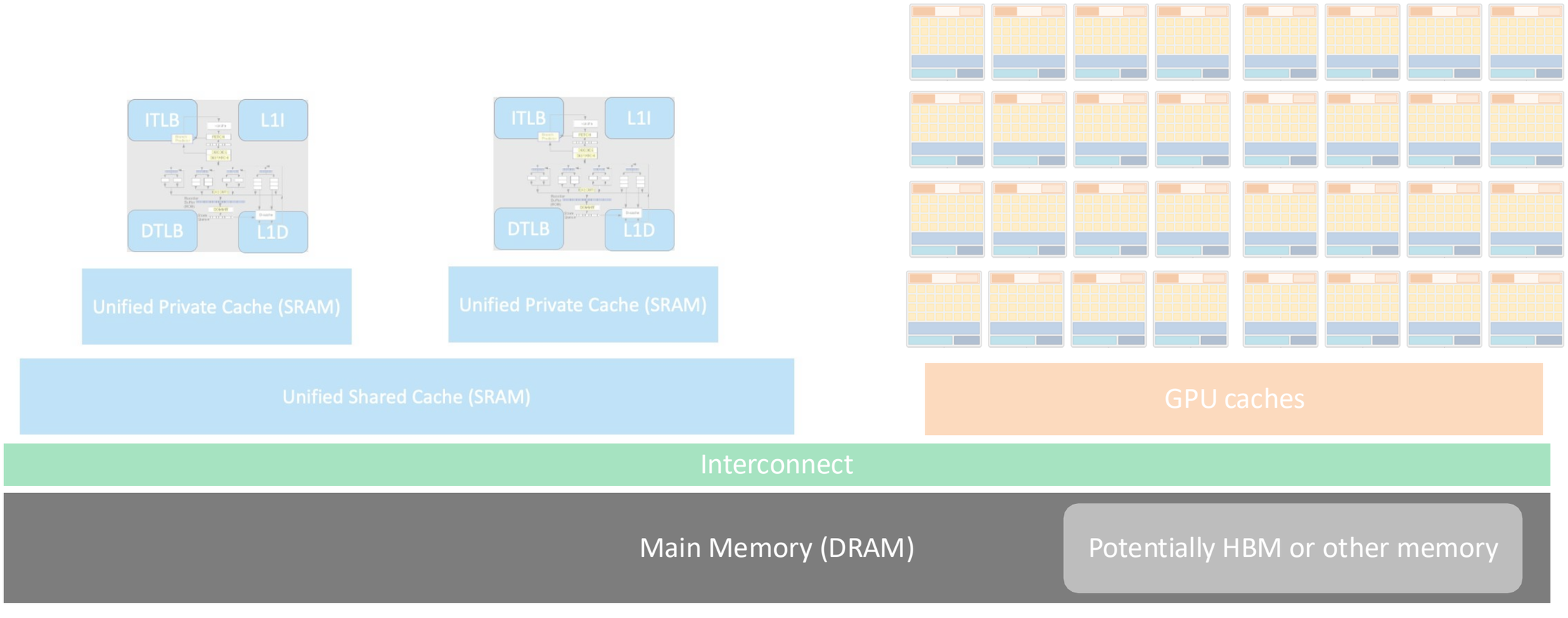

Showing a genericized version of the Lunar Lake cluster hierarchy (split L1s + TLBs, unified L2 & LLC) – Ignoring the L0 as the 18740 processor does not have an L0

# Memory Data Flow : Roadmap




Showing a genericized version of the Lunar Lake cluster hierarchy (split L1s + TLBs, unified L2 & LLC) – Ignoring the L0 as the 18740 processor does not have an L0

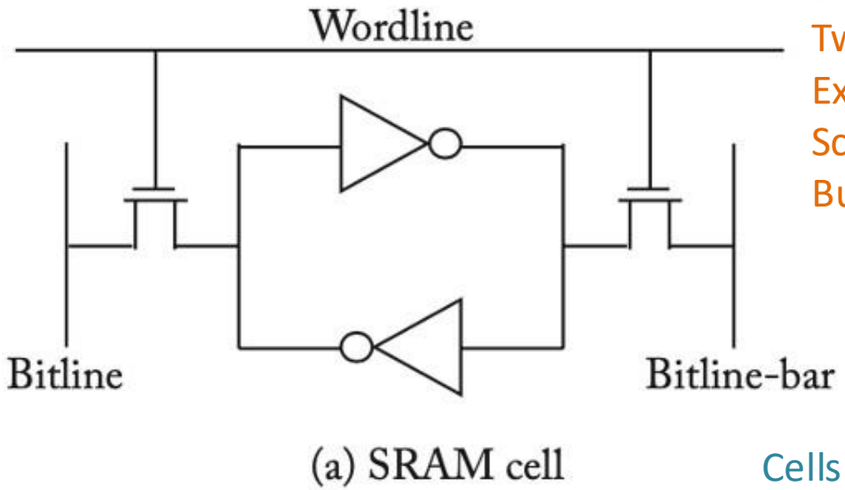
# Memory Data Flow : Main Memory




Showing a genericized version of the Lunar Lake cluster hierarchy (split L1s + TLBs, unified L2 & LLC) – Ignoring the L0 as the 18740 processor does not have an L0

# DRAM versus SRAM

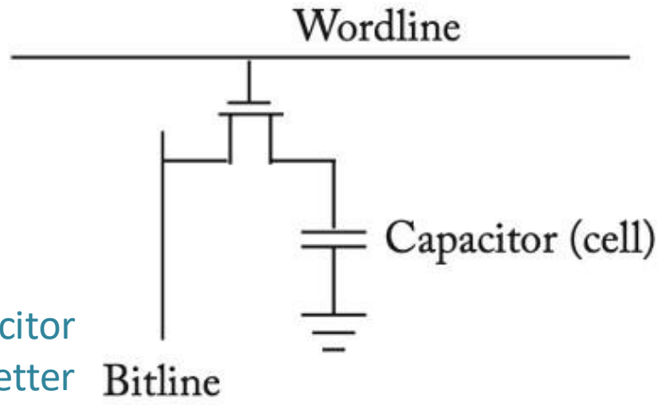
- Microarchitecture of a RAM is hierarchical and highly regular
  Optimized for density, yield (very competitive market with slim margins)
- An SRAM and DRAM differ in terms of cell circuitry



(a) SRAM cell

Bistable element (recall SR latch)
Two transistors per inverter = 6T
Expensive, low-density,
Scaling to smaller sizes is difficult
But no refresh, ***low latency***

A transistor and a capacitor
Much simpler, scales better
Cheap, low-density,
Cells continuously need refresh, ***high latency***

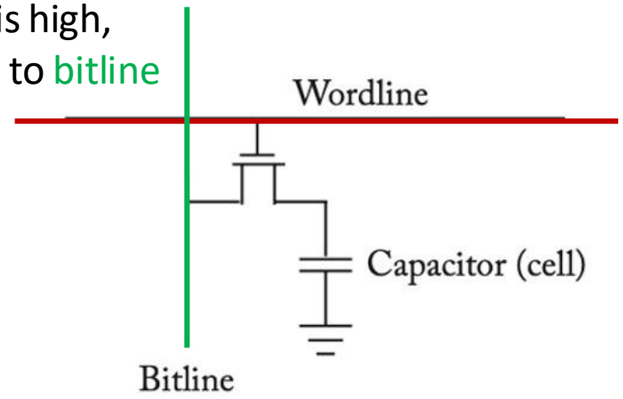


(b) DRAM cell

[1] "Innovations in the Memory Hierarchy", Balasubramonian Rajeev, Synthesis Lectures on Computer Architecture 2019

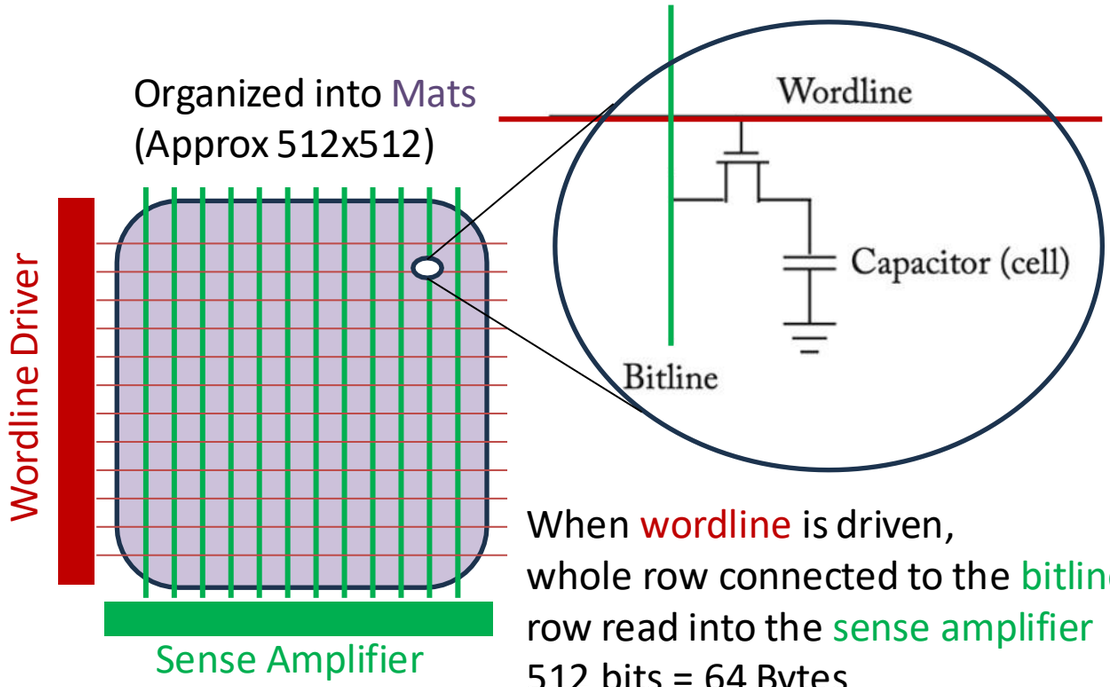
# DRAM organization

When wordline is high, capacitor "read" to bitline




destructive read, so need to "close page" afterwards

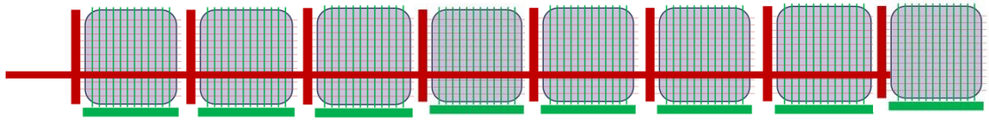
# DRAM organization



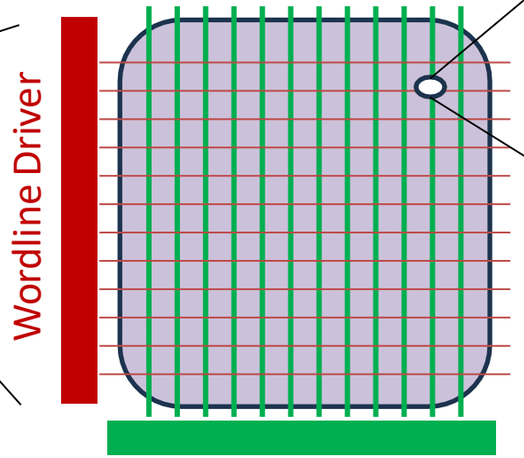

When wordline is driven,
whole row connected to the bitlines
row read into the sense amplifier
512 bits = 64 Bytes

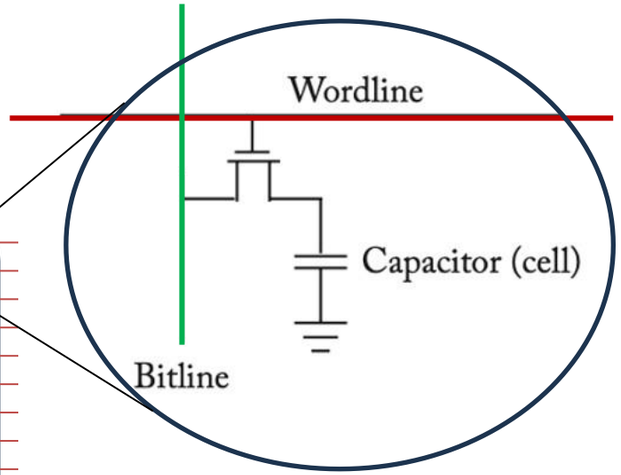
# DRAM organization

Arranged into Subarrays
(in this case, 8)



Share a common wordline (lockstep)
(entire row is 64B * 8 = 512B)

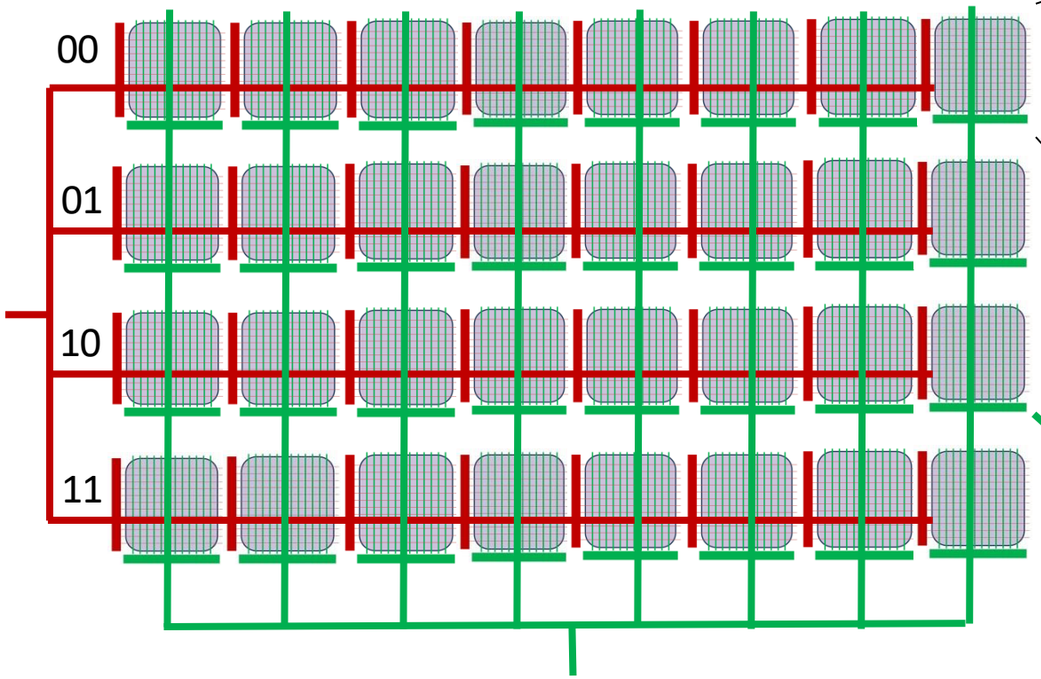


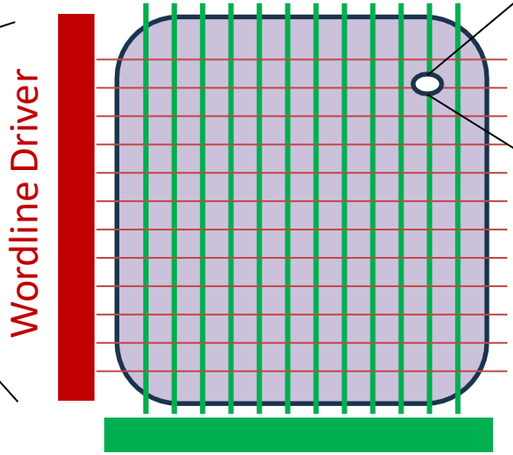



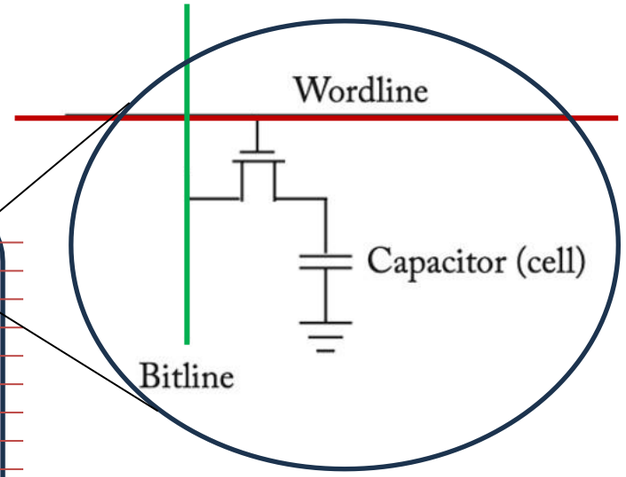
# DRAM organization

Subarrays form Banks

Wordlines driven based on address



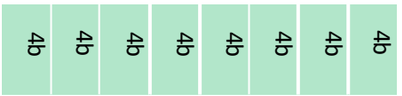




Conceptually – can view the activated subarray's sense amps as a single per-bank Row Buffer



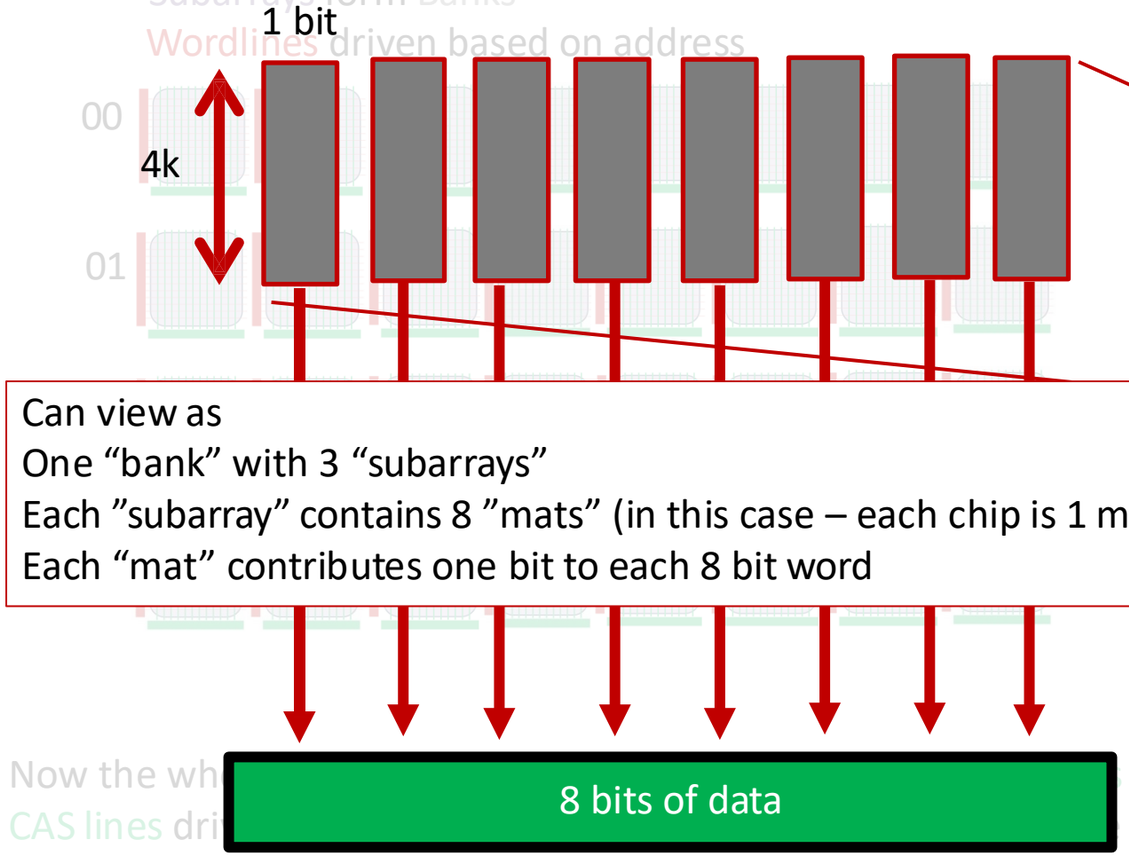

Now the whole 512B "page" is in that subarray's sense amplifiers

CAS lines driven to select which columns to take from each sense amplifier



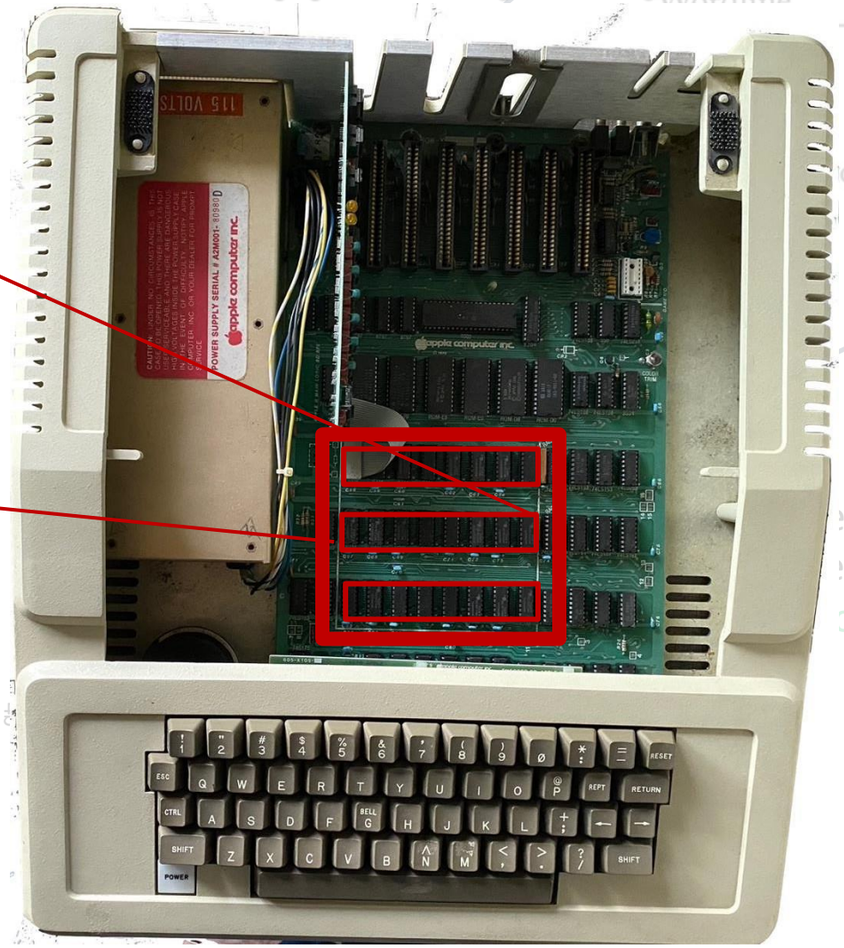

32B contiguous chunk of memory, striped across mats

# Minor digression…




Can view as
One “bank” with 3 “subarrays”
Each ”subarray” contains 8 ”mats” (in this case – each chip is 1 mat)
Each “mat” contributes one bit to each 8 bit word

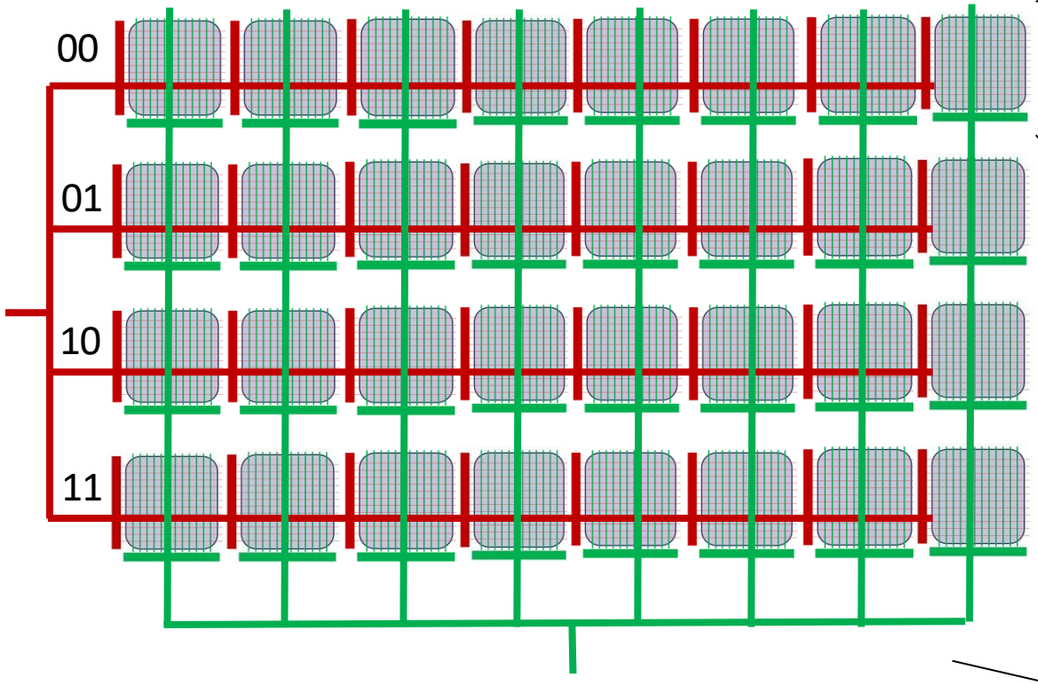
**Apple II+ (1979)**

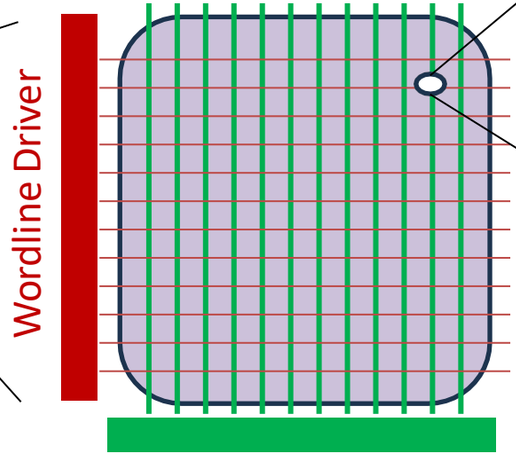


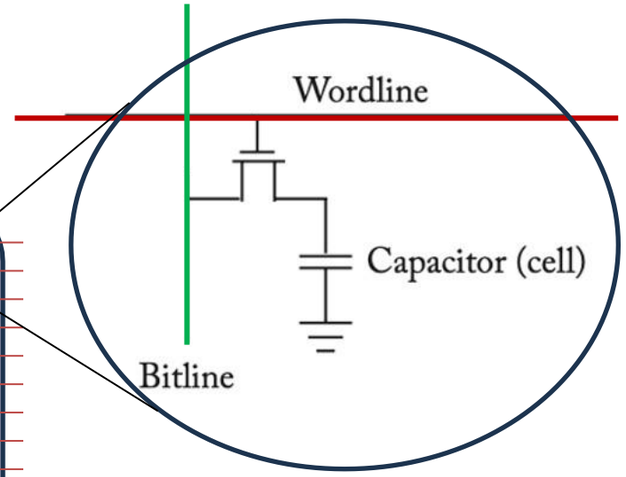
32B contiguous chunk of memory, striped across mats

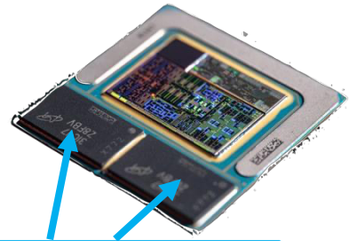
**More on a single chip – but organization has not significantly changed**

# DRAM organization

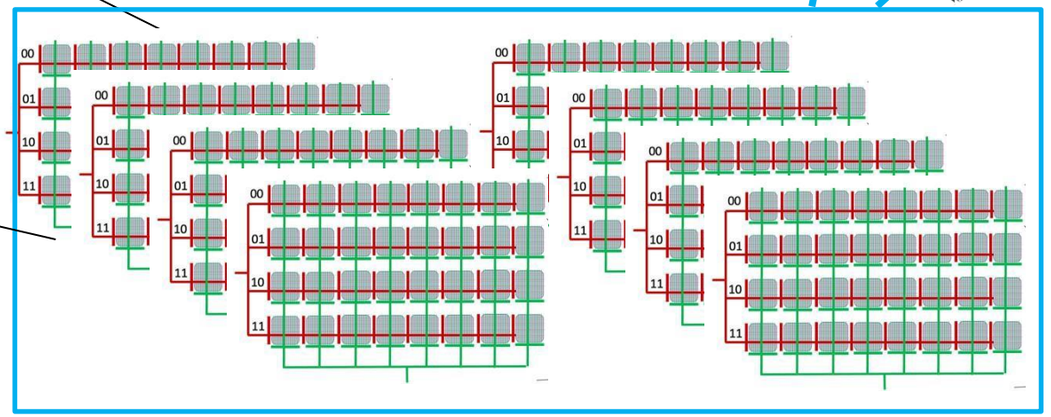


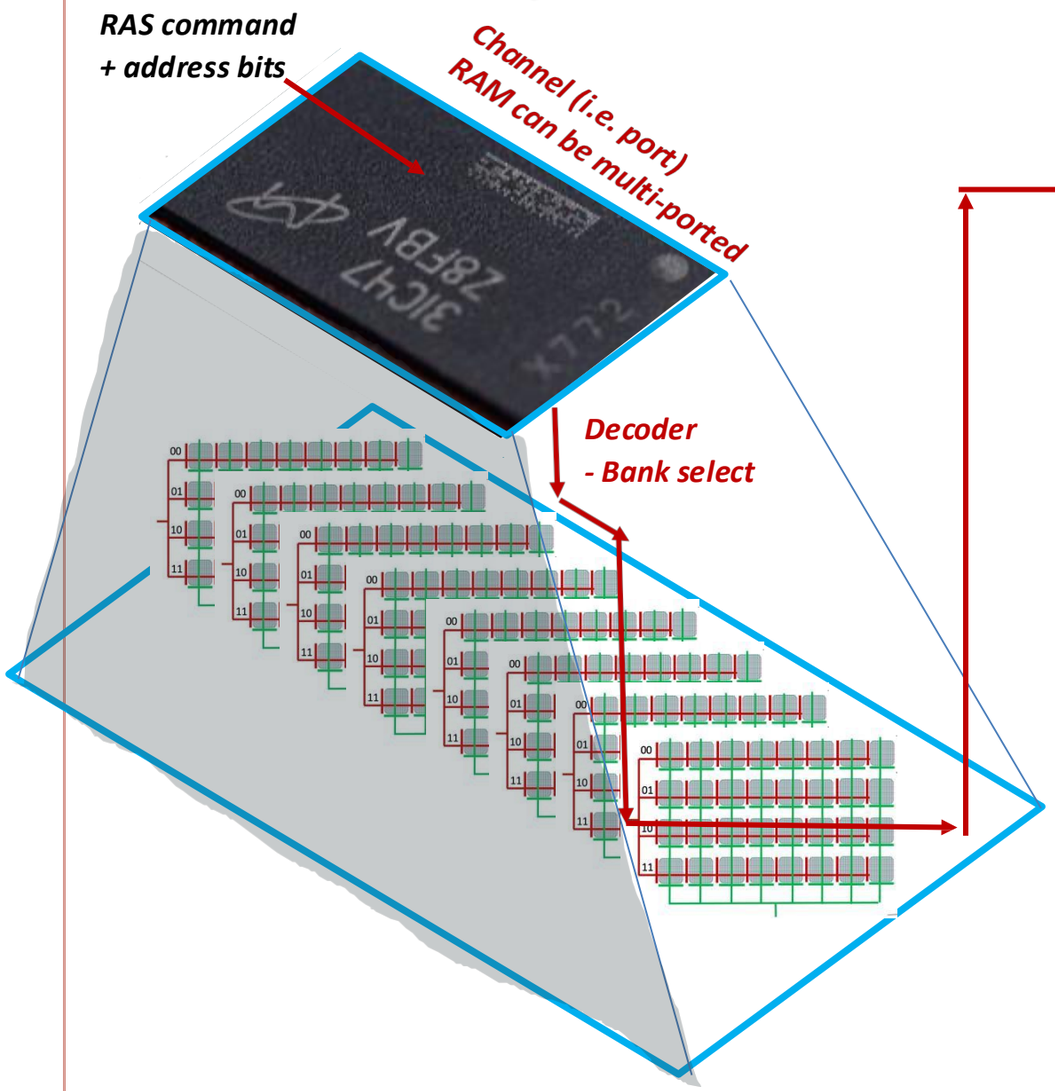



Chip is just multiple Banks



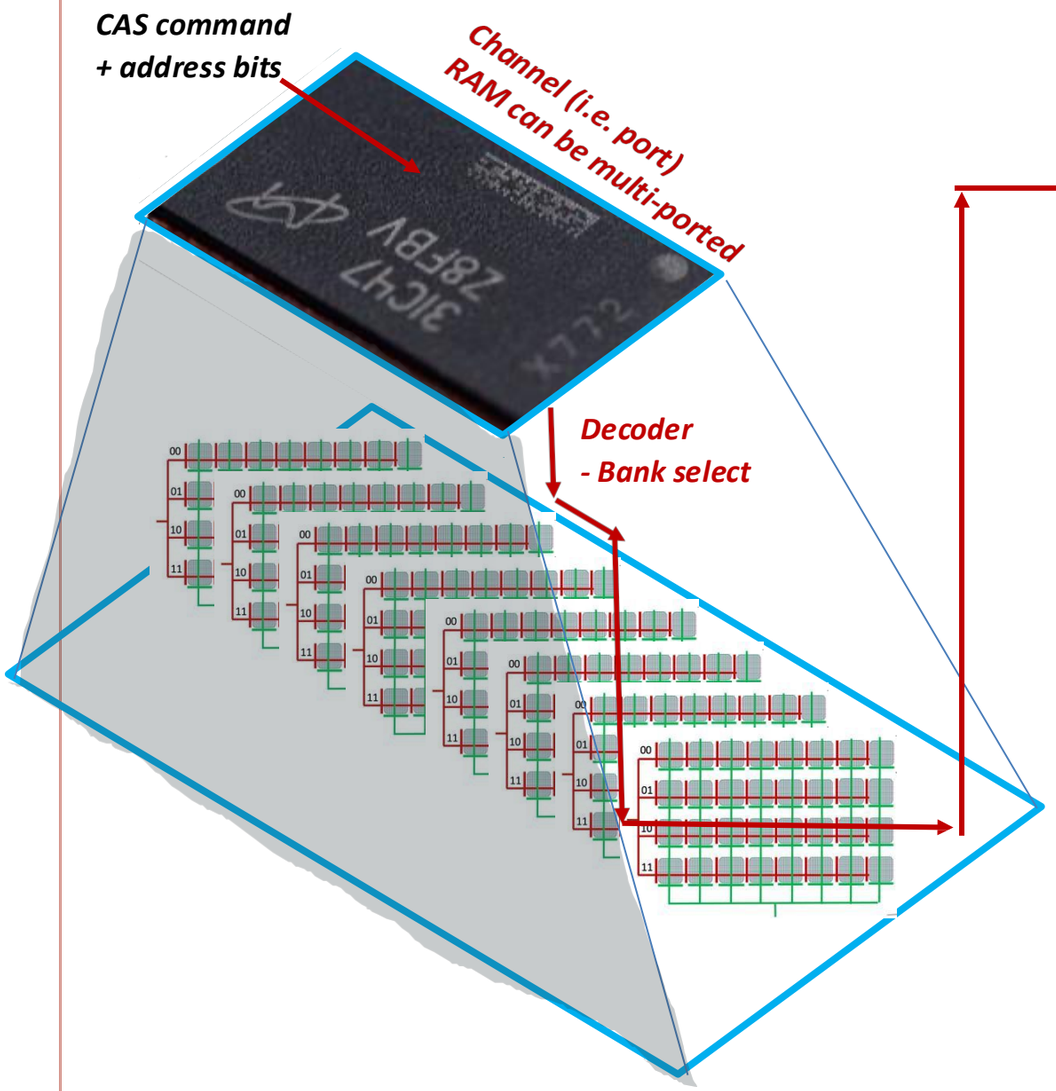
# Read operation : 1) RAS



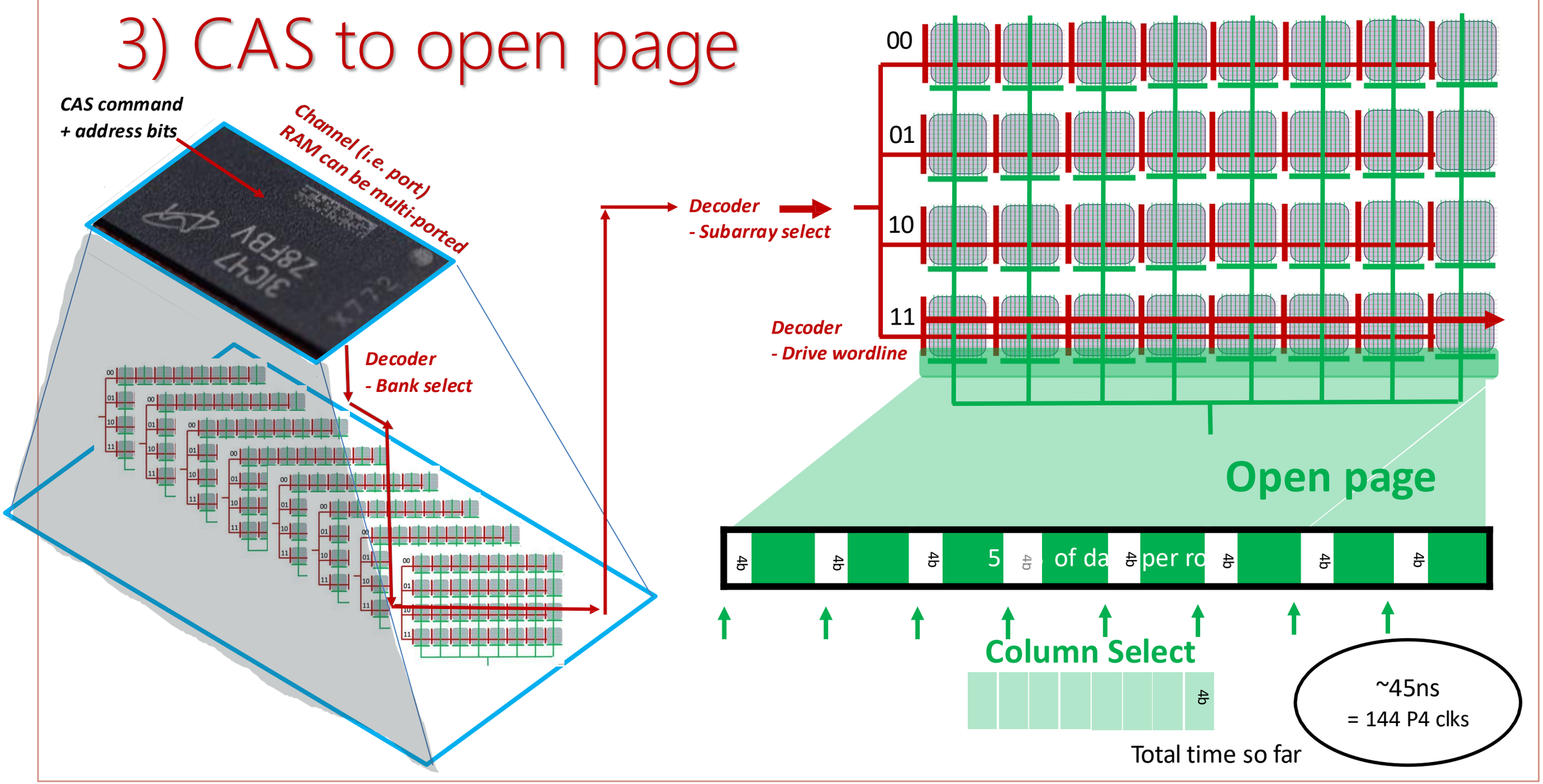

~15ns
= 48 P4 clks

Total time so far

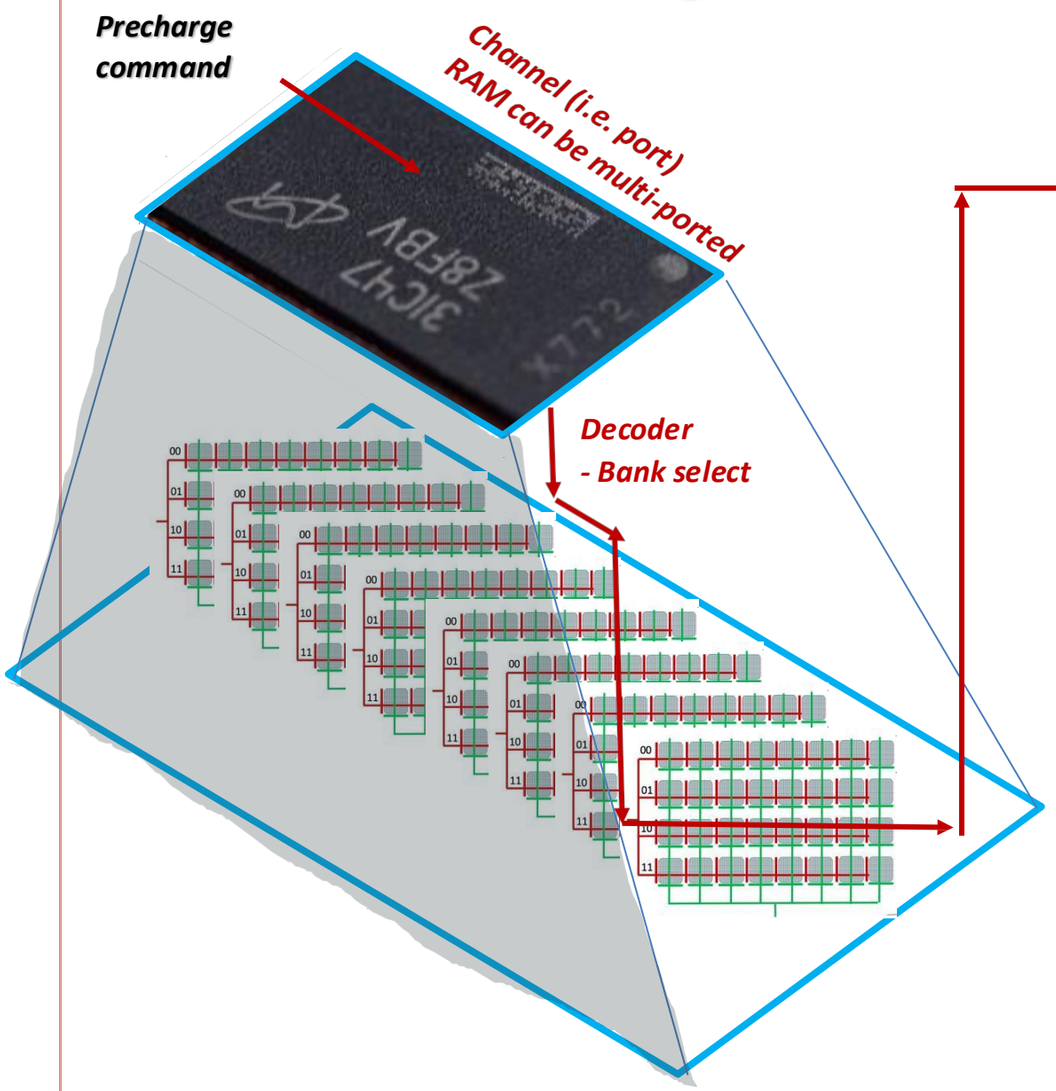
# 2) CAS

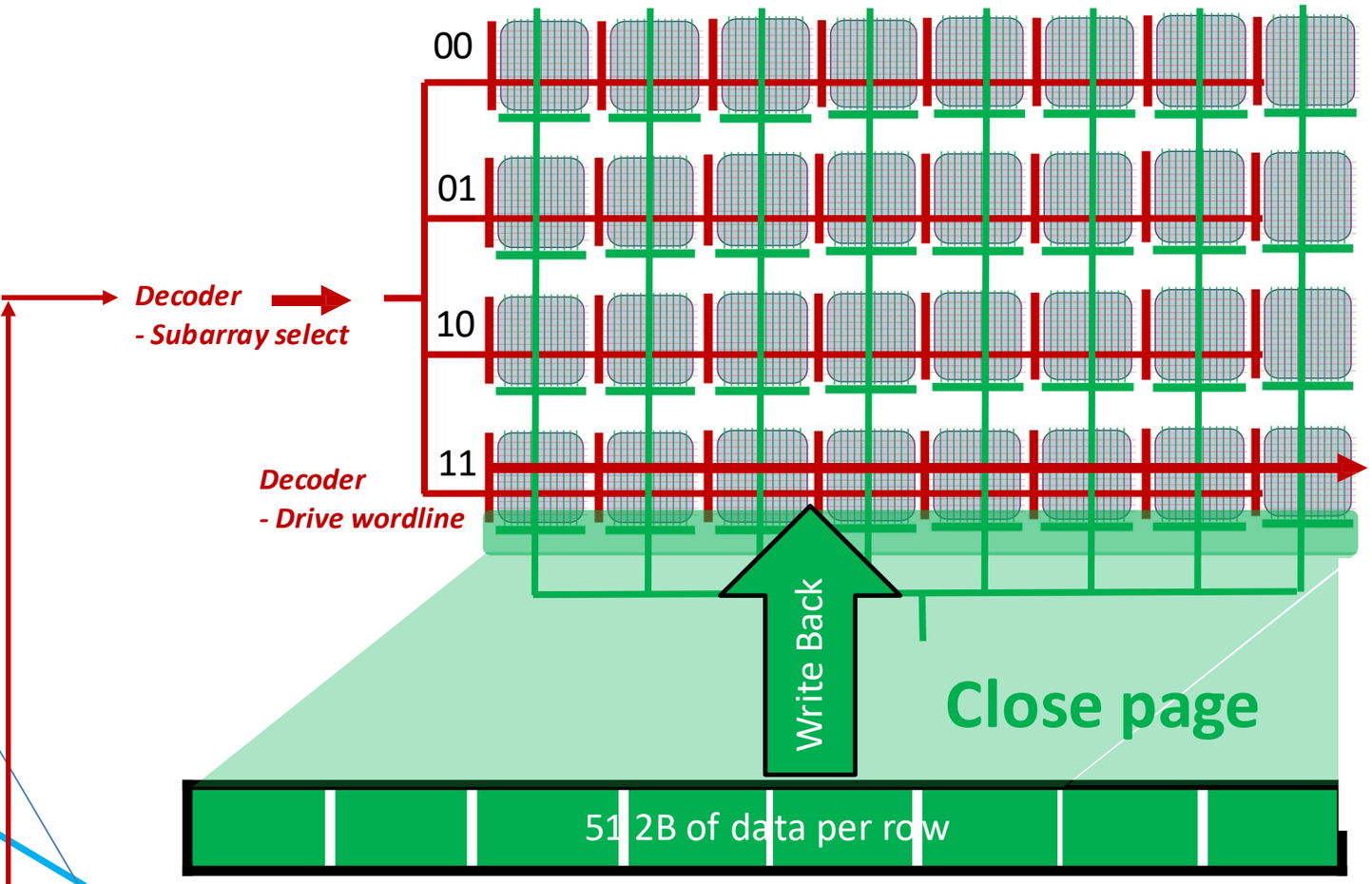

CAS command
+ address bits
Channel (i.e. port)
RAM can be multi-ported
Decoder
- Bank select
Decoder
- Subarray select
Decoder
- Drive wordline
00
01
10
11
Open page
4b
51 2
data
r
ro
w
Column Select
~30ns
= 96 P4 clks
Total time so far

# 3) CAS to open page



# 4) Precharge

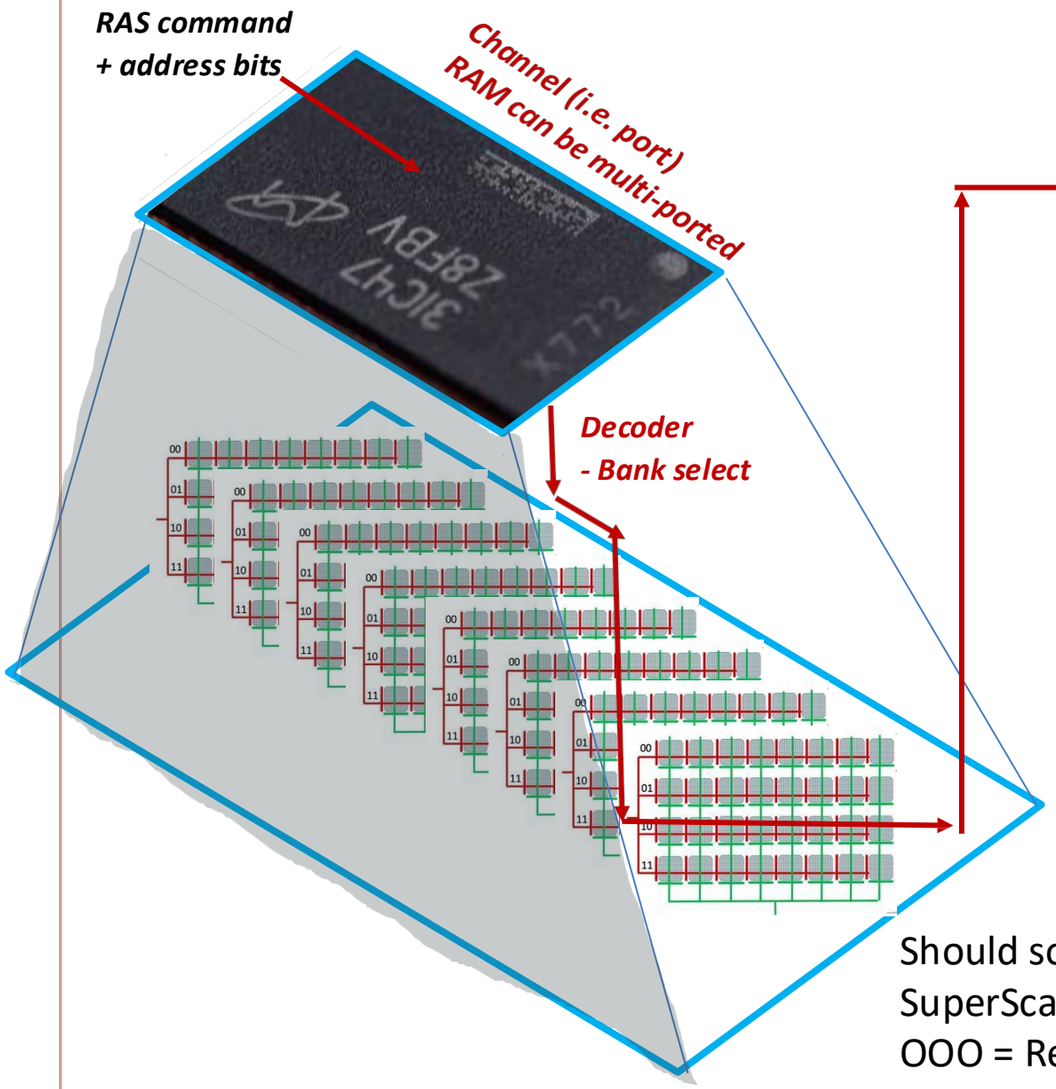


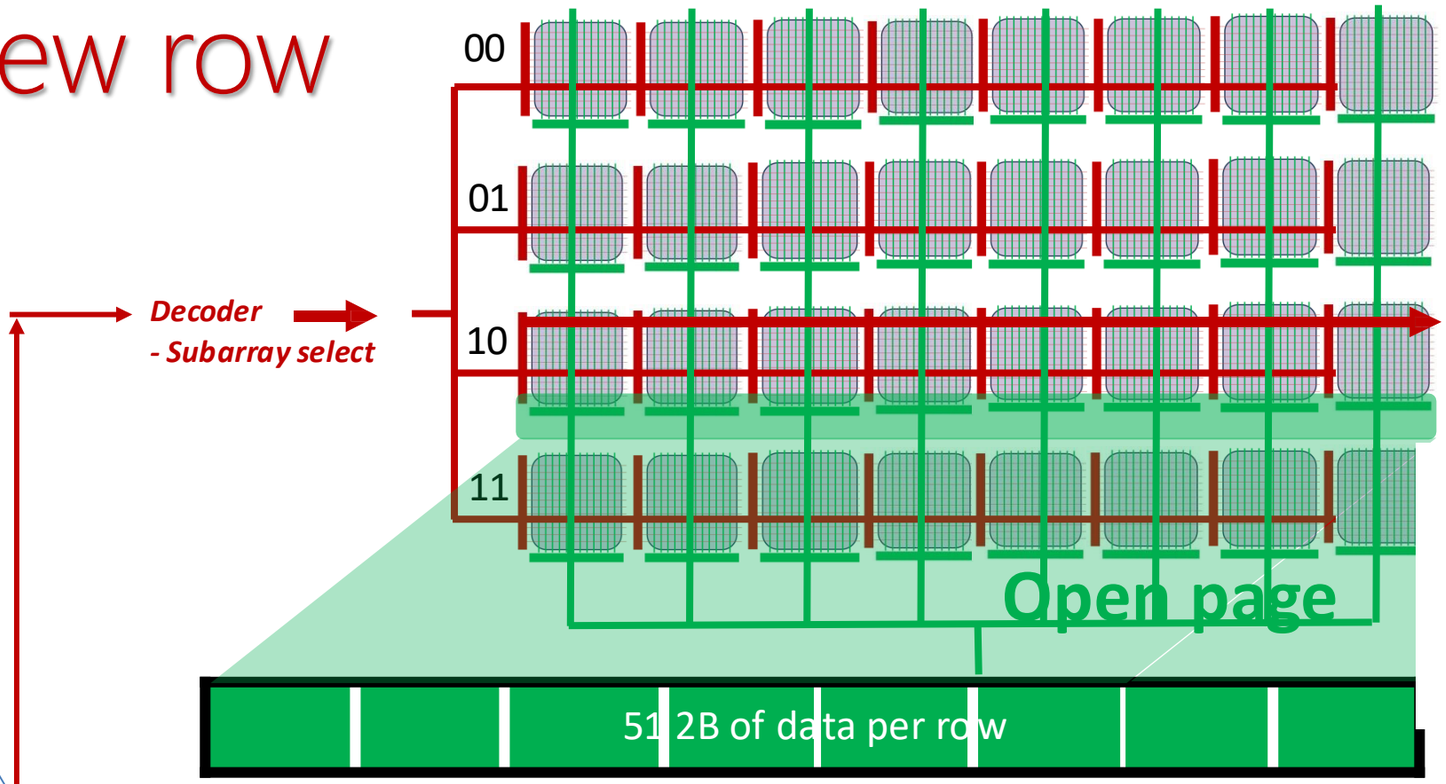




~60ns
= 192 P4 clks

Total time so far

# 5) RAS - open new row




Generally can sustain **one open row per-bank**
**Memory controller** tries to schedule commands efficiently

Should sound similar SS OOO CPU design…
SuperScalar = MultiChannel
OOO = Reordering Memory to maximize page-hit

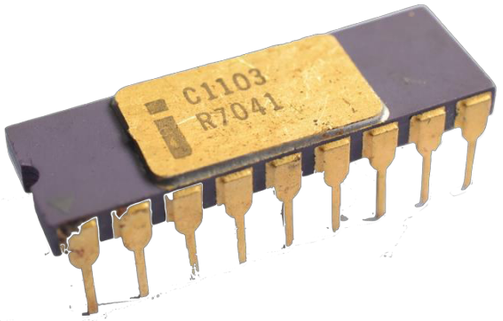
Total time so far

~75ns
= 240 P4 clks

# Memory controller design

- Complex hardware pipeline handling
  - Scheduling of requests from M agents onto N memory channels
  - Sophisticated algorithms to maximize page-hit and minimize page-close
  - Trying to avoid bank conflicts between channels
- Also responsible for handling ***Refresh***
  - DDR3 spec requires every cell be refreshed (i.e. precharge) 1/64ns
  - So, every row in every subarray in every bank must be read every 64ns…
  - Memory controller must explicitly provide these guarantees!
  - i.e. strategically choose good times to block requests to refresh
- In mixed DRAM/HBM or NUMA systems, there may be more than one

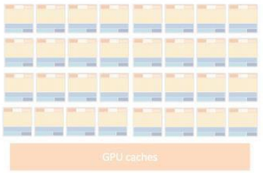
# Refresh granularity

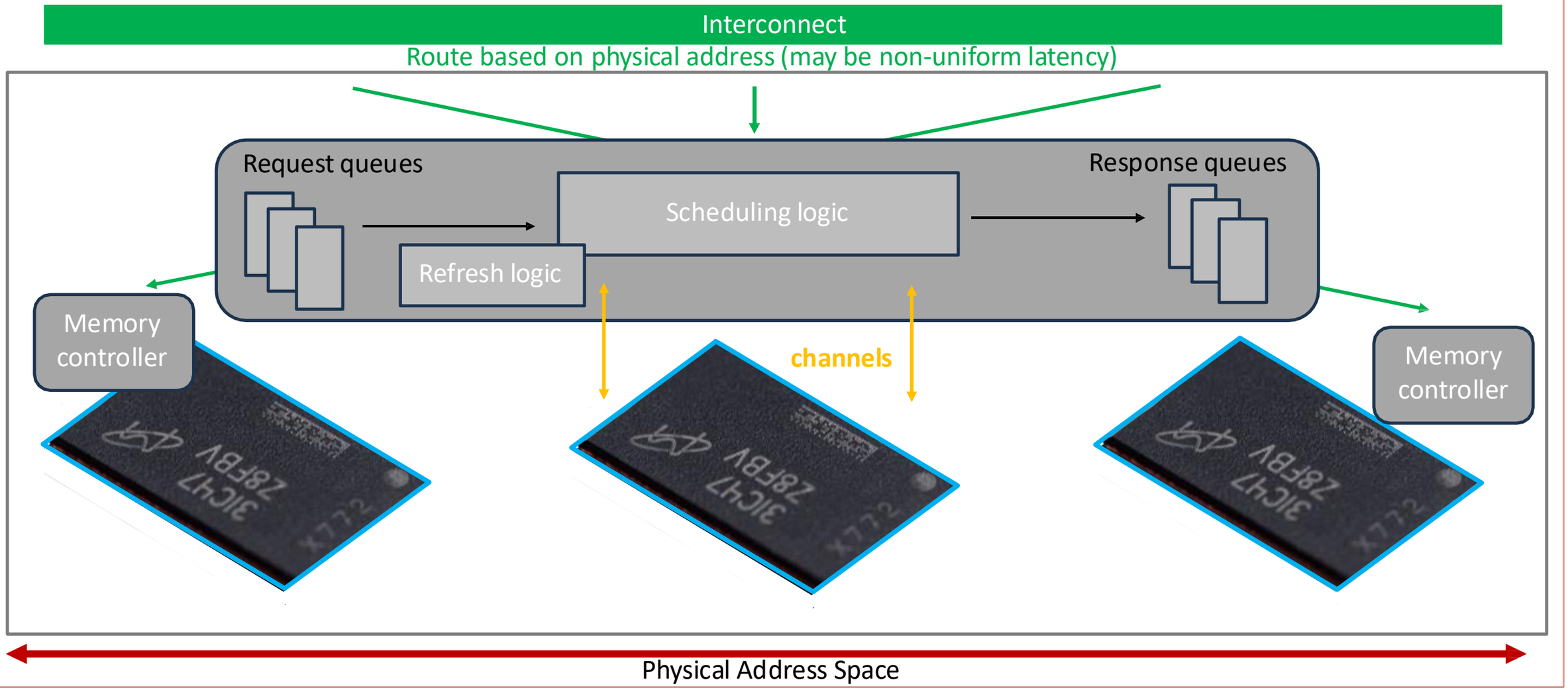


By Stelo.xyz, Pttn, or Thomas Nguyen, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=49532861

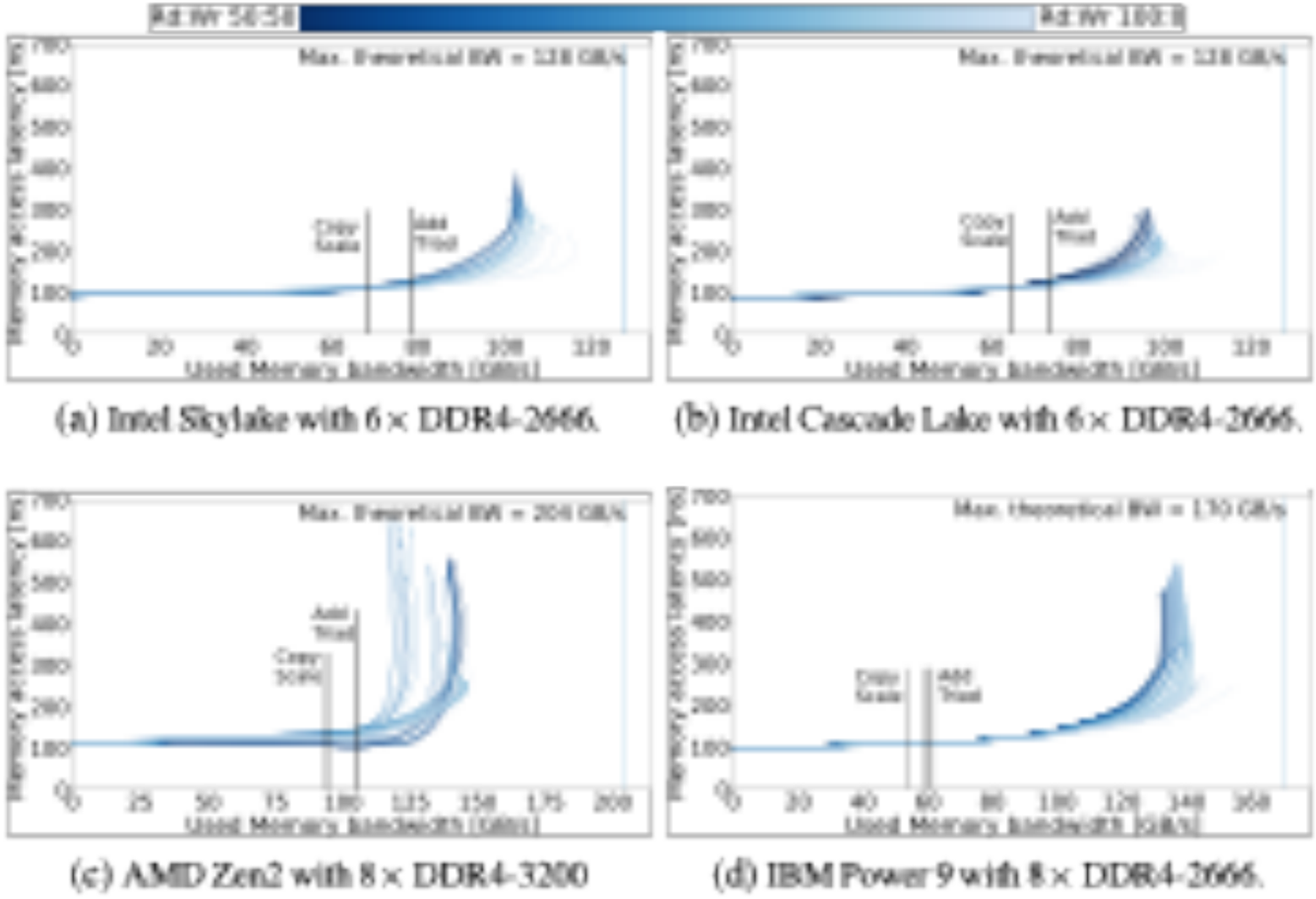
- Intel 1103 (1970)
  - 32 rows total
  - Memory controller (or CPU software) responsible for reading every row every 2ms
  - Either waste BW… or be clever about it…
- Modern DRAM can support coarse or fine-grained refresh
  - Coarse-grained is easier to design a memory controller for
    - (i.e. one command that blocks all of DRAM to refresh the entire thing)
  - But means high-performance memory controllers have less room for scheduling
    - (Middle ground might be to have bank-level or subarray-level refresh commands)
- **As DRAM shrinks, cells lose charge faster**

# A very generic memory controller





# Putting it together: DRAM characteristics



(a) Intel Skylake with 6× DDR4-2666. (b) Intel Cascade Lake with 6× DDR4-2666.

(c) AMD Zen2 with 8× DDR4-3200 (d) IBM Power 9 with 8× DDR4-2666.

"A Mess of Memory System Benchmarking, Simulation and Application Profiling" Esmaili-Dokht et al. MICRO 2024

# Scheduling policies

- First-come-first-serve
  - A “Closed-page” scheduling algorithm with opportunistic page-hits
- First-ready first-come-first-serve
  - FCFS but with minimal reordering in case there is a younger read request that would hit
- Refer to ISCA 2012 Memory Scheduling Championship for more

# Rowhammer mitigations & ECC

- DRAM typically manufactured with spare rows
  - In case yield is low… cells are faulty… etc…
- But DRAM errors **DO OCCUR**
  - **This is non-digital circuitry and is impacted by temperature, altitude, space rays**
  - Bit flips can be maliciously introduced by attacks such as Rowhammer attacks
- High-reliability RAMs can have error correcting codes built-in
  - Vary from error-detection to error-correction
  - Vary in terms of robustness to number of bit flips
- More specialized security features (like TRR) in newer specs

# Heterogeneous memory : HBM

- Optimized for high bandwidth above all else
  - Sacrifice density (which is still very important for AI), and perhaps latency
- Does this via two approaches
  - Enable more parallelism by routing more independent wires
    - Recall how all subarray wordlines operated in lockstep
    - Tradeoff is storage density
  - Stack a massive amount of DRAM dies on top of one & operate in parallel
    - Can't get this level of connectivity on a PCB… so 3D stacked
    - Giant through-silicon-vias enable high-bandwidth channels (say, 128bits)
    - Also a density tradeoff, and sometimes a frequency (latency) tradeoff
- **Doesn't count for much if you can't saturate BW**
  - Recall beginning of lecture... and perhaps it makes sense why this is more useful for GPU

# Memory compression

- For workloads with ***large*** memory footprints, can compress in memory
  - IBM MXT approach stores all data as compressed in RAM
  - Memory controller is responsible for converting PA to true location in compressed space
  - Memory controller is responsible for compressing/decompressing @ channel interface
- Proposed approaches to leak compression further up the stack
  - Cache compression is a play to increase virtual capacity of caches
    - Something that a memory-latency-bound workload would much appreciate
- Proposed approaches to employ compression for cache-to-memory links
  - I.e. compress packets over the interconnect
- Specialized compression algorithms exist – refer to synthesis lecture

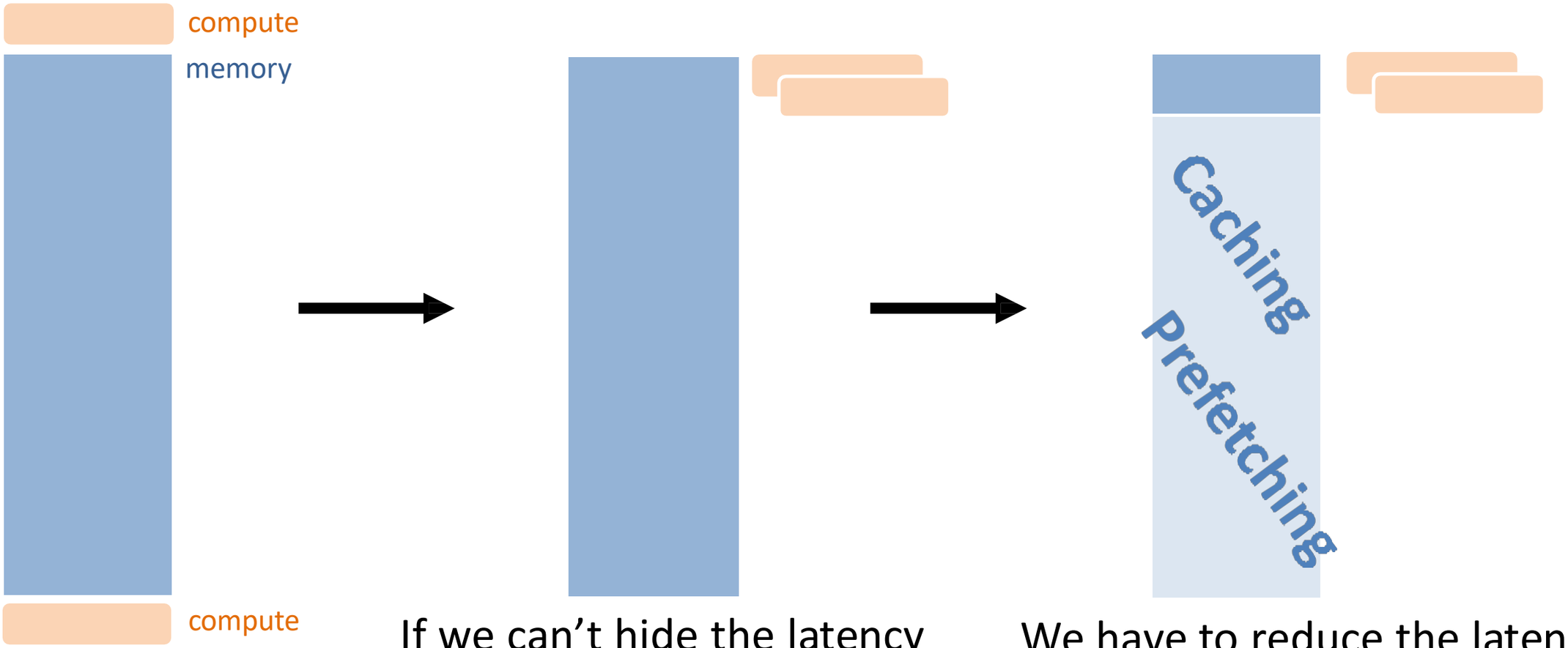
[1] "A Primer on Compression in the Memory Hierarchy", Sardashti et al, Synthesis Lectures on Computer Architecture 2015

# Near/In-memory computing

- Has been a research topic for a ***long time***
- Toy example:
  - A CPU adds Mem[A] and Mem[B] to store result in Mem[C]
  - Load Mem[A], Mem[B] all the way from memory, write Mem[C] all the way back
  - Could we instead just send the “add” command to memory?
- Problems
  - Density – compute layers would be taking up storage capacity
  - Doesn’t make sense for all operations
    - Do you want your add instruction to take 960 cycles?
    - If the memory is in-cache, what do you do?
- But… work continues (SK Hynix Accelerator-in-Memory Hotchips ‘24)



# CPU as a "Latency Machine"




If we can't hide the latency

We have to reduce the latency

**Deeper cache hierarchies – smart replacement policies – high-coverage prefetchers**

# Many Memory Data Prefetching Ideas

- Next Line, Stride Based
- Target Delinquent Loads
- Speculative Precomputation
- Dynamic Helper Threads
- Slice-Based Prefetch Threads

# 18-740: Modern Computer Architecture & Design

## Lecture 9:
## "System Level Memory Prefetching"

**Tyler Huberty**

September 22, 2025

**Next Time ...**

- Required Reading Assignment:
  - **Chapter 3 of Shen and Lipasti (SnL).**
  - From **Synthesis lectures of Computer Architecture**
    - Sec. 1 & 3: Bruce Jacob, "The Memory System: You Can't Avoid It, You Can't Ignore It, You Can't Fake It," 2009.
    - Sec. 1 & 4: Balasubramonian Rajeev, "Innovations in the Memory System," 2019.

Electrical & Computer ENGINEERING